Implementation Plan

SnowReach — Neighbourhood Snow Clearing Marketplace

Version: 1.0 **Date:** 2026-04-02 **Specification source:** SPECIFICATION.md v1.0 **Development Plan source:** DEVELOPMENT_PLAN.md v1.0 **Status:** Active

> **This is a living document.** Update task status in TASK_TRACKER.html as work progresses.

How to Use This Document

Each task section contains four parts:

1. **Dependencies** — tasks that must be complete before this one begins
2. **Tools & Libraries** — what needs to be installed or available
3. **Assumptions** — what is taken as true going in
4. **AI Implementation Prompt** — copy-paste this into Claude Code (or a similar AI assistant) to generate the code for the task

Before using any prompt: prepend the **Context Preamble** below. It gives the AI the project context it needs to produce consistent, on-spec code.

Tasks are numbered ENV-01 through P3-08. Work them in dependency order. The ENV tasks must all be complete before any Phase 0 code tasks begin.

Context Preamble

> **Copy this block and paste it before every AI Implementation Prompt.**

PROJECT: SnowReach — a neighbourhood snow-clearing marketplace connecting Requesters (property owners needing snow clearing) with Workers (snowblower owners) in Ontario, Canada.

TECH STACK:
- Frontend: React 18 + Vite + React Router v6, CSS Modules
- Backend: Java 21 + Spring Boot 3.x + Maven
- Database: Firebase Firestore (primary operational DB + real-time messaging/notifications)
- Auth: Firebase Auth (email/password + Google + Apple OAuth)
- File storage: Firebase Storage
- Frontend hosting: Firebase Hosting
- Backend hosting: Google Cloud Run
- Payments: Stripe Payment Intents (escrow) + Stripe Connect Express (Worker payouts)
- Geolocation: Google Maps Geocoding API — server-side only, key never sent to browser

- Email: SendGrid
- Scheduling: Quartz Scheduler with H2 in-memory store (Phase 1)
- Audit log: separate Firebase Firestore project, append-only, SHA-256 hash chaining
- Version control: Git + GitHub

FILE STRUCTURE ROOT: snowreach/
  frontend/ — React/Vite app
  backend/ — Spring Boot app (package com.snowreach)
  firebase/ — Firebase config files (firestore.rules, storage.rules, firebase.json)
  .github/workflows/ — CI/CD pipelines

KEY BUSINESS RULES:
- 15% platform commission on all Worker payouts
- Full escrow: funds held by platform until job COMPLETE + rated, or dispute resolved
- 13% Ontario HST on all transactions, passed through 100% to Worker (not subject to commission)
- Workers ranked by rating DESC then distance ASC; sequential dispatch; one active job at a time (Phase 1)
- $10 CAD cancellation fee if cancelled after CONFIRMED but before IN_PROGRESS
- 2-hour dispute window after COMPLETE; 2–3 business day payouts after RELEASED
- All Firestore writes go through Spring Boot (Firebase Admin SDK); client never writes operational data
- Real-time reads (messages, notifications feed) use Firestore client SDK directly from React
- RBAC: roles[] array in Firestore users/{uid}; mirrored to Firebase Auth custom claims
  Roles: REQUESTER, WORKER, ADMIN

JOB STATE MACHINE (11 states):
  REQUESTED → PENDING_DEPOSIT → CONFIRMED → IN_PROGRESS → COMPLETE → RELEASED
  Also: INCOMPLETE, DISPUTED (from COMPLETE) → RELEASED or REFUNDED, SETTLED, CANCELL

AUDIT LOG: AuditLogService.write() must be called BEFORE every state-changing Firestore write.
Hash chain: SHA-256(previousHash + timestamp + actorUid + action + entityId + JSON(before) + JSON(afte

CONVENTIONS:
- REST API base path: /api
- Auth header: Authorization: Bearer {firebaseIdToken}
- Error responses: RFC 7807 Problem JSON {type, title, status, detail}
- All monetary amounts stored as integer cents (CAD)
- Timestamps: Firestore Timestamp objects; ISO-8601 strings in API responses
- Java: constructor injection, no field injection; services are @Service; controllers are @RestController
- React: functional components only; no class components; hooks for all state

---

ENV-01 — Install Core Development Tools

Type: Environment Setup | **Must be done first**

Dependencies

- None — this is the first step.

Tools to Install

Tool	Version	Source
Node.js	20 LTS	nodejs.org
Java (Eclipse Temurin)	21 LTS	adoptium.net
Apache Maven	3.9+	maven.apache.org
Git	Latest	git-scm.com
VS Code	Latest	code.visualstudio.com
Google Cloud CLI	Latest	cloud.google.com/sdk
Firebase CLI	Latest (via npm)	—
Stripe CLI	Latest	stripe.com/docs/stripe-cli

Step-by-Step Instructions

1. **Node.js v20 LTS** - Download the Windows installer from https://nodejs.org → "20.x.x LTS" - Run installer; accept defaults; ensure "Add to PATH" is checked - Verify: open a new terminal and run: node -v # should show v20.x.x npm -v # should show 10.x.x

2. **Java 21 (Eclipse Temurin)** - Go to https://adoptium.net → select Java 21, Windows x64 MSI - Run installer; check "Set JAVA_HOME variable" and "Add to PATH" - Verify: java -version # should show openjdk version "21.x.x" echo %JAVA_HOME% # should show install path - If JAVA_HOME is not set automatically: System Properties → Advanced → Environment Variables → New System Variable: JAVA_HOME = C:\Program Files\Eclipse Adoptium\jdk-21.x.x-hotspot

3. **Apache Maven 3.9+** - Download binary zip from https://maven.apache.org/download.cgi - Extract to C:\Program Files\Apache\maven-3.9.x - Add to PATH: System Properties → Environment Variables → Edit Path → Add C:\Program Files\Apache\maven-3.9.x\bin - Verify: mvn -version # should show Apache Maven 3.9.x, Java 21

4. **Git** - Download from https://git-scm.com/download/win - Run installer; recommended options: "Git from the command line and also from 3rd-party software"; default editor: VS Code; "Use Windows' default console window" - Configure identity (required before first commit): git config --global user.name "Your Name" git config --global user.email "your@email.com" git config --global core.autocrlf true - Verify: git --version

5. **VS Code** - If not installed: download from https://code.visualstudio.com - During install, check: "Add to PATH", "Register Code as editor for supported file types" - Verify: code --version

6. **Google Cloud CLI** - Download installer from https://cloud.google.com/sdk/docs/install (Windows x86_64) - Run installer; accept defaults - After install, run: gcloud init - Sign in with your Google account - Select or create the GCP project for SnowReach - Verify: gcloud version - Install additional components: gcloud components install beta

7. **Firebase CLI**

```
npm install -g firebase-tools
firebase login      # opens browser for Google auth
firebase --version  # should show 13.x.x or later
```

8. **Stripe CLI** - Download from https://github.com/stripe/stripe-cli/releases → Windows zip - Extract stripe.exe to a folder on your PATH (e.g. C:\Program Files\Stripe\) - Add that folder to PATH via System Properties - Authenticate: stripe login (opens browser) - Verify: stripe --version

Verification Checklist

Run all of the following — each must succeed before proceeding to ENV-02:

```
node -v && npm -v
java -version
mvn -version
git --version
code --version
gcloud version
firebase --version
stripe --version
```

Troubleshooting

- **mvn not found:** PATH not updated; restart terminal or run refreshenv (if using Chocolatey)
- **Java version shows 11 or 17:** Multiple JDKs installed; update JAVA_HOME to point to Temurin 21
- **firebase login fails:** Ensure you're logged into the right Google account; try firebase logout first
- **gcloud init fails:** Ensure the Google account has Owner or Editor role on the GCP project

---

ENV-02 — Create Project Folder and Initialize Git

Type: Environment Setup

Dependencies

- ENV-01 complete (Git installed and configured)

Step-by-Step Instructions

1. The snowreach folder

The folder C:\Users\perel\claude_code\snowreach\ already exists and contains the project documents (REQUIREMENTS.md, SPECIFICATION.md, etc.). Do not create a new folder — initialize git inside the existing one.

```
cd C:\Users\perel\claude_code\snowreach
git init
git branch -M main
```

2. Create .gitignore

Create C:\Users\perel\claude_code\snowreach\.gitignore with the following content:

```
# ── Node / Frontend ──────────────────────────────────────────────
node_modules/
frontend/dist/
frontend/.env
frontend/.env.local
frontend/.env.*.local
npm-debug.log*
yarn-debug.log*
yarn-error.log*

# ── Java / Backend ───────────────────────────────────────────────
backend/target/
backend/.mvn/wrapper/maven-wrapper.jar
!backend/.mvn/wrapper/maven-wrapper.properties
*.class
*.jar
*.war
*.nar
*.ear
*.zip
*.tar.gz
*.rar

# ── Spring Boot ──────────────────────────────────────────────────
backend/src/main/resources/firebase-service-account.json
backend/src/main/resources/firebase-service-account-audit.json
backend/.env
```

backend/application-secrets.yml

```
# ── Firebase ──────────────────────────────────────────────────────────────
.firebase/
firebase-debug.log
firestore-debug.log
ui-debug.log
.firebaserc.local

# ── Quartz / H2 ───────────────────────────────────────────────────────────
*.mv.db
*.trace.db
quartz.properties

# ── IDE / Editor ──────────────────────────────────────────────────────────
.idea/
*.iml
*.iws
*.ipr
.classpath
.project
.settings/
.vscode/settings.json

# ── OS ────────────────────────────────────────────────────────────────────
.DS_Store
.DS_Store?
._*
.Spotlight-V100
.Trashes
ehthumbs.db
Thumbs.db
desktop.ini

# ── Secrets / credentials ─────────────────────────────────────────────────
*.pem
*.key
*.p12
*.jks
.env
.env.*
!.env.example
secrets/
credentials/

# ── Logs ──────────────────────────────────────────────────────────────────
```

```
logs/
*.log
```

3. Initial commit

```
git add .gitignore README.md REQUIREMENTS.md SPECIFICATION.md DEVELOPMENT_PLAN.md IMPL
git commit -m "chore: initialize SnowReach repository with project documents"
```

4. Verify

```
git log --oneline    # should show 1 commit
git status           # should show nothing to commit
```

ENV-03 — Create GitHub Repository and Link Remote

Type: Environment Setup

Dependencies

- ENV-02 complete (local git initialized, first commit made)
- A GitHub account with access to create repositories

Step-by-Step Instructions

1. Create the GitHub repository - Go to https://github.com/new - Repository name: snowreach - Visibility: **Private** - Do NOT initialize with README, .gitignore, or license (we already have these locally) - Click “Create repository”

2. Link and push

```
git remote add origin https://github.com/YOUR_USERNAME/snowreach.git
git push -u origin main
```

3. Create develop branch

```
git checkout -b develop
git push -u origin develop
git checkout main
```

4. Create the GitHub Actions workflow folder

```
mkdir -p .github\workflows
```

Create placeholder files (will be filled in during P1-02): - .github/workflows/frontend-deploy.yml — content: # Frontend CI/CD — configured in P1-02 - .github/workflows/backend-deploy.yml — content: # Backend CI/CD — configured in P1-02

Commit and push:

```
git add .github/
git commit -m "chore: add CI/CD workflow placeholders"
git push
```

5. Configure branch protection (GitHub UI) - Go to repo → Settings → Branches → Add rule - Branch name pattern: main - Check: "Require a pull request before merging" - Check: "Require approvals" (set to 1) - Check: "Do not allow bypassing the above settings" - Save

6. Add GitHub Secrets (Settings → Secrets and variables → Actions → New repository secret)

Add placeholders now; values will be filled in as each service is set up:

Secret Name	Used By	Set In
FIREBASE_SERVICE_ACCOUNT	backend-deploy.yml	P1-01
FIREBASE_PROJECT_ID	both workflows	P1-01
GOOGLE_CLOUD_PROJECT	backend-deploy.yml	P1-02
STRIPE_SECRET_KEY	backend at runtime	P1-11
STRIPE_WEBHOOK_SECRET	backend at runtime	P1-11
SENDGRID_API_KEY	backend at runtime	P1-17
MAPS_API_KEY	backend at runtime	P1-07
CERTN_API_KEY	backend at runtime	P3-01

ENV-04 — VS Code Setup and Extensions

Type: Environment Setup

Dependencies

- ENV-01 complete (VS Code installed)
- ENV-03 complete (repo cloned or folder open in VS Code)

Step-by-Step Instructions

1. Open the project in VS Code

```
code C:\Users\perel\claude_code\snowreach
```

2. Install extensions

Open the Extensions panel (Ctrl+Shift+X) and install each of the following by searching the marketplace ID:

Extension	Marketplace ID	Purpose
Extension Pack for Java	vscjava.vscode-java-pack	Java language support, debugger, test runner
Spring Boot Tools	vmware.vscode-spring-boot	Spring Boot-aware navigation and hints
Spring Initializr	vscjava.vscode-spring-initializr	Scaffold new Spring Boot projects
ESLint	dbaeumer.vscode-eslint	JavaScript/React linting
Prettier	esbenp.prettier-vscode	Code formatting
ES7+ React Snippets	dsznajder.es7-react-js-snippets	React component snippets
GitLens	eamodio.gitlens	Git blame, history, and branch management
GitHub Pull Requests	github.vscode-pull-request-github	Review PRs inside VS Code
Firebase Explorer	toba.vsfire	Browse Firestore and Storage in VS Code
Auto Rename Tag	formulahendry.auto-rename-tag	Rename paired HTML/JSX tags simultaneously
Path Intellisense	christian-kohler.path-intellisense	Autocomplete file paths in imports
DotENV	mikestead.dotenv	Syntax highlighting for .env files
REST Client	humao.rest-client	Send HTTP requests from .http files
Markdown All in One	yzhang.markdown-all-in-one	Preview and shortcuts for Markdown
Code Spell Checker	streetsidesoftware.code-spell-checker	Catch typos in code and comments

3. Create .vscode/extensions.json

```
{
  "recommendations": [
    "vscjava.vscode-java-pack",
    "vmware.vscode-spring-boot",
    "vscjava.vscode-spring-initializr",
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "dsznajder.es7-react-js-snippets",
    "eamodio.gitlens",
    "github.vscode-pull-request-github",
    "toba.vsfire",
    "formulahendry.auto-rename-tag",
```

```
    "christian-kohler.path-intellisense",
    "mikestead.dotenv",
    "humao.rest-client",
    "yzhang.markdown-all-in-one",
    "streetsidesoftware.code-spell-checker"
  ]
}
```

4. Create .vscode/settings.json

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "[java]": {
    "editor.defaultFormatter": "redhat.java",
    "editor.tabSize": 4
  },
  "[javascript]": { "editor.tabSize": 2 },
  "[javascriptreact]": { "editor.tabSize": 2 },
  "[css]": { "editor.tabSize": 2 },
  "[json]": { "editor.tabSize": 2 },
  "java.configuration.runtimes": [
    { "name": "JavaSE-21", "path": "C:\\Program Files\\Eclipse Adoptium\\jdk-21.x.x-hotspot", "default": true }
  ],
  "java.compile.nullAnalysis.mode": "automatic",
  "spring-boot.ls.problem.application-properties.unknown-property": "WARNING",
  "files.exclude": {
    "**/node_modules": true,
    "**/backend/target": true,
    "**/.firebase": true
  },
  "editor.rulers": [100],
  "files.trimTrailingWhitespace": true,
  "files.insertFinalNewline": true,
  "eslint.workingDirectories": ["frontend"],
  "prettier.configPath": "frontend/.prettierrc"
}
```

Note: .vscode/settings.json is in .gitignore so local settings won't be committed. The extensions.json file IS committed so teammates get the same extension recommendations.

5. Configure Java runtime path

Update the java.configuration.runtimes path in settings.json to match your actual Temurin 21 install path. Find it with:

```
where java
```

6. **Verify Java works in VS Code** - Open any .java file (will create one in ENV-05) - The Java Language Server should start automatically (status bar bottom) - Wait for “Java: Ready” in the status bar before proceeding

---

ENV-05 — Establish Complete File Structure

Type: Environment Setup

Dependencies

- ENV-01 through ENV-04 complete

Overview

Create the complete directory tree and all skeleton files. Directories with a * next to them will be populated by later tasks — create them now so the structure is visible and IDE navigation works from day one.

Complete Directory Tree

```
snowreach/
├── .github/
│   └── workflows/
│       ├── backend-deploy.yml      ← placeholder from ENV-03
│       └── frontend-deploy.yml     ← placeholder from ENV-03
├── .vscode/
│   ├── extensions.json             ← from ENV-04
│   └── settings.json               ← from ENV-04 (gitignored)
├── backend/
│   ├── pom.xml                     ← skeleton (filled in P1-03)
│   └── src/
│       ├── main/
│       │   ├── java/com/snowreach/
│       │   │   ├── SnowReachApplication.java
│       │   │   ├── config/
│       │   │   │   ├── CorsConfig.java
│       │   │   │   ├── FirebaseConfig.java
│       │   │   │   ├── QuartzConfig.java
│       │   │   │   └── SecurityConfig.java
│       │   │   ├── controller/
│       │   │   │   ├── AdminController.java
│       │   │   │   ├── DisputeController.java
```

```
│   │   │   │   ├── JobController.java
│   │   │   │   ├── PaymentController.java
│   │   │   │   ├── RatingController.java
│   │   │   │   ├── UserController.java
│   │   │   │   ├── WebhookController.java
│   │   │   │   └── WorkerController.java
│   │   │   ├── dto/
│   │   │   │   ├── CreateJobRequest.java
│   │   │   │   ├── CreateUserRequest.java
│   │   │   │   ├── DisputeRequest.java
│   │   │   │   ├── RatingRequest.java
│   │   │   │   ├── UpdateJobRequest.java
│   │   │   │   └── WorkerProfileRequest.java
│   │   │   ├── exception/
│   │   │   │   ├── GlobalExceptionHandler.java
│   │   │   │   ├── InvalidTransitionException.java
│   │   │   │   ├── JobNotFoundException.java
│   │   │   │   └── PaymentException.java
│   │   │   ├── model/
│   │   │   │   ├── AuditEntry.java
│   │   │   │   ├── Dispute.java
│   │   │   │   ├── Job.java
│   │   │   │   ├── Notification.java
│   │   │   │   ├── Rating.java
│   │   │   │   ├── User.java
│   │   │   │   └── Worker.java
│   │   │   ├── scheduler/
│   │   │   │   ├── DispatchJob.java
│   │   │   │   └── DisputeTimerJob.java
│   │   │   ├── security/
│   │   │   │   ├── FirebaseTokenFilter.java
│   │   │   │   └── RbacInterceptor.java
│   │   │   ├── service/
│   │   │   │   ├── AuditLogService.java
│   │   │   │   ├── DispatchService.java
│   │   │   │   ├── DisputeService.java
│   │   │   │   ├── GeocodingService.java
│   │   │   │   ├── JobService.java
│   │   │   │   ├── MatchingService.java
│   │   │   │   ├── NotificationService.java
│   │   │   │   ├── PaymentService.java
│   │   │   │   ├── RatingService.java
```

```
│   │   │   │   ├── UserService.java
│   │   │   │   └── WorkerService.java
│   │   │   └── util/
│   │   │       ├── GeoUtils.java
│   │   │       └── HashUtils.java
│   │   └── resources/
│   │       ├── application-dev.yml
│   │       ├── application-prod.yml
│   │       └── application.yml
│   └── test/
│       └── java/com/snowreach/
│           ├── JobServiceTest.java
│           ├── MatchingServiceTest.java
│           └── StateMachineTest.java
├── docs/
│   ├── architecture.md
│   └── runbook.md
├── firebase/
│   ├── .firebaserc
│   ├── firebase.json
│   ├── firestore.indexes.json
│   ├── firestore.rules
│   └── storage.rules
├── frontend/
│   ├── .eslintrc.cjs
│   ├── .prettierrc
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   ├── public/
│   │   └── favicon.ico          ← placeholder (16x16 white square is fine)
│   └── src/
│       ├── App.jsx
│       ├── main.jsx
│       ├── mockData.js
│       ├── router.jsx
│       ├── assets/
│       │   └── logo.svg
│       ├── components/
│       │   ├── Badge/
│       │   │   └── Badge.jsx
│       │   ├── Button/
```

```
│   │   │   ├── Button.jsx
│   │   │   └── Button.module.css
│   │   ├── Card/
│   │   │   ├── Card.jsx
│   │   │   └── Card.module.css
│   │   ├── Input/
│   │   │   ├── Input.jsx
│   │   │   └── Input.module.css
│   │   ├── Modal/
│   │   │   ├── Modal.jsx
│   │   │   └── Modal.module.css
│   │   ├── Spinner/
│   │   │   └── Spinner.jsx
│   │   └── StatusPill/
│   │       └── StatusPill.jsx
│   ├── context/
│   │   ├── AuthContext.jsx
│   │   ├── MockStateContext.jsx
│   │   └── NotificationContext.jsx
│   ├── hooks/
│   │   ├── useAuth.js
│   │   ├── useJob.js
│   │   └── useNotifications.js
│   ├── layouts/
│   │   ├── AdminLayout.jsx
│   │   ├── RequesterLayout.jsx
│   │   └── WorkerLayout.jsx
│   ├── pages/
│   │   ├── admin/
│   │   │   ├── Analytics.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   └── JobDetail.jsx
│   │   ├── auth/
│   │   │   ├── Login.jsx
│   │   │   └── Signup.jsx
│   │   ├── requester/
│   │   │   ├── Home.jsx
│   │   │   ├── JobStatus.jsx
│   │   │   ├── PostJob.jsx
│   │   │   ├── RateWorker.jsx
│   │   │   └── WorkerProfile.jsx
│   │   └── worker/
```

```
│       │       ├── ActiveJob.jsx
│       │       ├── Earnings.jsx
│       │       ├── JobRequest.jsx
│       │       └── Register.jsx
│       ├── services/
│       │   ├── api.js
│       │   ├── firebase.js
│       │   └── stripe.js
│       ├── styles/
│       │   ├── globals.css
│       │   └── tokens.css
│       └── utils/
│           ├── constants.js
│           ├── formatCurrency.js
│           └── formatDate.js
├── .gitignore
├── DECISIONS_LOG.md
├── DEVELOPMENT_PLAN.md
├── IMPLEMENTATION_PLAN.md
├── README.md
├── REQUIREMENTS.md
├── SPECIFICATION.md
├── TASK_TRACKER.html
└── USE_CASES.md
```

Commands to Create Structure

Run from C:\Users\perel\claude_code\snowreach:

```
# Backend directories
mkdir -p backend/src/main/java/com/snowreach/config
mkdir -p backend/src/main/java/com/snowreach/controller
mkdir -p backend/src/main/java/com/snowreach/dto
mkdir -p backend/src/main/java/com/snowreach/exception
mkdir -p backend/src/main/java/com/snowreach/model
mkdir -p backend/src/main/java/com/snowreach/scheduler
mkdir -p backend/src/main/java/com/snowreach/security
mkdir -p backend/src/main/java/com/snowreach/service
mkdir -p backend/src/main/java/com/snowreach/util
mkdir -p backend/src/main/resources
mkdir -p backend/src/test/java/com/snowreach

# Frontend directories
mkdir -p frontend/public
```

```
mkdir -p frontend/src/assets
mkdir -p frontend/src/components/Badge
mkdir -p frontend/src/components/Button
mkdir -p frontend/src/components/Card
mkdir -p frontend/src/components/Input
mkdir -p frontend/src/components/Modal
mkdir -p frontend/src/components/Spinner
mkdir -p frontend/src/components/StatusPill
mkdir -p frontend/src/context
mkdir -p frontend/src/hooks
mkdir -p frontend/src/layouts
mkdir -p frontend/src/pages/admin
mkdir -p frontend/src/pages/auth
mkdir -p frontend/src/pages/requester
mkdir -p frontend/src/pages/worker
mkdir -p frontend/src/services
mkdir -p frontend/src/styles
mkdir -p frontend/src/utils

# Firebase and docs
mkdir -p firebase
mkdir -p docs
```

Skeleton File Contents

Create each file below. Files marked **[placeholder]** get a single comment line; files marked **[skeleton]** get meaningful starter content.

backend/src/main/java/com/snowreach/SnowReachApplication.java [skeleton]:

```java
package com.snowreach;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SnowReachApplication {
    public static void main(String[] args) {
        SpringApplication.run(SnowReachApplication.class, args);
    }
}
```

All other Java files: [placeholder] — one line: package com.snowreach.XXX; // TODO: implement in task PX-XX

backend/src/main/resources/application.yml [skeleton]:

```yaml
spring:
  application:
    name: snowreach-api
  profiles:
    active: ${SPRING_PROFILE:dev}

server:
  port: 8080

management:
  endpoints:
    web:
      exposure:
        include: health
```

backend/src/main/resources/application-dev.yml [skeleton]:

```yaml
# Dev overrides — filled in P1-03
logging:
  level:
    com.snowreach: DEBUG
```

backend/src/main/resources/application-prod.yml [skeleton]:

```yaml
# Prod overrides — filled in P1-23
logging:
  level:
    com.snowreach: INFO
```

frontend/src/main.jsx [skeleton]:

```jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'
import './styles/globals.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

frontend/src/App.jsx [skeleton]:

```jsx
export default function App() {
  return <div>SnowReach — coming soon</div>
}
```

All other .jsx / .js files in frontend/: [placeholder] — // TODO: implement in task PX-XX

firebase/firestore.rules [skeleton]:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // DENY ALL — security rules implemented in P1-21
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

firebase/storage.rules [skeleton]:

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    // DENY ALL — storage rules implemented in P1-21
    match /{allPaths=**} {
      allow read, write: if false;
    }
  }
}
```

firebase/firebase.json [skeleton]:

```
{
  "hosting": {
    "public": "../frontend/dist",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [{ "source": "**", "destination": "/index.html" }]
  },
  "firestore": {
    "rules": "firestore.rules",
    "indexes": "firestore.indexes.json"
  },
  "storage": {
    "rules": "storage.rules"
  }
}
```

firebase/.firebaserc [skeleton]:

```
{
  "projects": {
    "default": "YOUR_FIREBASE_PROJECT_ID"
  }
}
```

firebase/firestore.indexes.json [skeleton]:

```
{
  "indexes": [],
  "fieldOverrides": []
}
```

docs/architecture.md [placeholder]: # Architecture — see SPECIFICATION.md §2

docs/runbook.md [placeholder]: # On-Call Runbook — populated in P2-08

frontend/src/styles/tokens.css [placeholder]: /* Design tokens — implemented in P0-02 */

frontend/src/styles/globals.css [placeholder]: /* Global styles — implemented in P0-02 */

frontend/src/mockData.js [placeholder]: // Mock data for Phase 0 prototype — populated in P0-01

Commit the Structure

```
git add .
git commit -m "chore: establish complete project file structure"
git push
```

P0-01 — React Project Setup

Phase: 0 — UI Prototype | **Week:** W1 | **Depends on:** ENV-01 through ENV-05

Dependencies

- Node.js 20 LTS installed (ENV-01)
- frontend/ directory created (ENV-05)

Tools & Libraries

Tool / Library	Version	Purpose
Vite	5.x	Build tool and dev server
React	18.x	UI framework
React DOM	18.x	DOM rendering
React Router	6.x	Client-side routing
ESLint	8.x	Code linting
eslint-plugin-react	7.x	React-specific lint rules
Prettier	3.x	Code formatting

Assumptions

- The frontend/ directory exists but is currently empty (only skeleton files from ENV-05)
- All React work uses functional components and hooks exclusively — no class components
- Routing uses React Router v6 with <Outlet /> pattern for nested layouts
- CSS Modules are used for component-scoped styles; global styles in src/styles/
- No TypeScript in Phase 0; TypeScript migration is a post-MVP consideration
- The app will have three distinct role-based layouts: Requester, Worker, Admin

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Scaffold the SnowReach React frontend (Phase 0 — UI Prototype).

Working directory: snowreach/frontend/

The frontend/ directory already exists with placeholder files. Set up a complete Vite + React 18 project inside it.

DO THE FOLLOWING:

1. Create package.json with these exact dependencies:
   - react ^18.3.0, react-dom ^18.3.0
   - react-router-dom ^6.22.0
   devDependencies:
   - vite ^5.2.0, @vitejs/plugin-react ^4.2.0
   - eslint ^8.57.0, eslint-plugin-react ^7.34.0, eslint-plugin-react-hooks ^4.6.0
   - prettier ^3.2.0
   Scripts: dev, build, preview, lint, format

2. Create vite.config.js:
   - React plugin enabled
   - Dev server on port 3000
   - Proxy /api requests to http://localhost:8080 (for future backend integration)

3. Create index.html (in frontend/ root, not src/):
   - charset UTF-8, viewport meta
   - Title: "SnowReach"
   - Link to /src/main.jsx as module script
   - id="root" div

4. Create src/main.jsx:
   - Import React, ReactDOM
   - Import BrowserRouter from react-router-dom
   - Import App
   - Import ./styles/globals.css
   - Render <BrowserRouter><App /></BrowserRouter> into #root

5. Create src/router.jsx:
   Define ALL routes using createBrowserRouter or nested <Routes>. Include routes for:
   Requester role (under /requester/):
     - /requester/ → RequesterLayout outlet
       - index → pages/requester/Home
       - post-job → pages/requester/PostJob
       - jobs/:jobId → pages/requester/JobStatus
       - jobs/:jobId/rate → pages/requester/RateWorker
       - workers/:workerId → pages/requester/WorkerProfile
   Worker role (under /worker/):
     - /worker/ → WorkerLayout outlet
       - index → pages/worker/Earnings (dashboard home)
       - register → pages/worker/Register
       - job-request/:requestId → pages/worker/JobRequest
       - active-job → pages/worker/ActiveJob
   Admin role (under /admin/):
     - /admin/ → AdminLayout outlet
       - index → pages/admin/Dashboard
       - jobs/:jobId → pages/admin/JobDetail
       - analytics → pages/admin/Analytics
   Auth:
     - /login → pages/auth/Login
     - /signup → pages/auth/Signup
   Root:
     - / → redirect to /requester/ (prototype default)
   All layout and page components can be empty placeholder components for now
   (they will be filled in later tasks).

6. Update src/App.jsx:
   Import and render the router (using RouterProvider if using createBrowserRouter,
   or wrap routes in Router). This should be minimal — just routing setup.

7. Create .eslintrc.cjs:
   - env: browser, es2021
   - extends: eslint:recommended, plugin:react/recommended, plugin:react-hooks/recommended
   - parserOptions: ecmaVersion latest, sourceType module
   - rules: react/prop-types off (no PropTypes in Phase 0), no-unused-vars warn,
     react/react-in-jsx-scope off (React 17+ auto-import)

8. Create .prettierrc:
```
{
  "semi": false,
  "singleQuote": true,
  "tabWidth": 2,
  "trailingComma": "es5",
  "printWidth": 100
}
```

9. Create placeholder layout files (just export a div with outlet):
   src/layouts/RequesterLayout.jsx — renders <Outlet />, has a nav bar placeholder
   src/layouts/WorkerLayout.jsx — renders <Outlet />, has a nav bar placeholder
   src/layouts/AdminLayout.jsx — renders <Outlet />, has a sidebar placeholder

10. Create placeholder page files for every route listed above — each just returns
    PageName — coming soon

After scaffolding, the dev server must start with `npm run dev` with zero errors.
Run `npm run lint` and fix all lint errors before finishing.
Provide the complete content of all files.

P0-02 — Design System

Phase: 0 — UI Prototype | **Week:** W1 | **Depends on:** P0-01

Dependencies

- P0-01 complete (Vite + React project running)

Tools & Libraries

Tool / Library	Version	Purpose
CSS Custom Properties	Native	Design tokens
Inter font	Via Google Fonts CDN	Primary typeface

Assumptions

- All design tokens are CSS custom properties on :root in tokens.css
- Component styles use CSS Modules (.module.css) importing from tokens
- No external UI library (MUI, Tailwind, etc.) — custom design system only
- Mobile-first: base styles target 375px, media queries scale up
- Brand: SnowReach blue family + neutral grays + semantic colours

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement the SnowReach design system (P0-02).

Files to create/update:
- frontend/src/styles/tokens.css
- frontend/src/styles/globals.css
- frontend/index.html (add Google Fonts link)

1. tokens.css — define ALL CSS custom properties on :root:

COLOURS:
```
  --color-primary:       #1A6FDB   (SnowReach blue)
  --color-primary-dark:   #0F4FA8
  --color-primary-light:  #E8F1FC
  --color-snow:          #F0F6FF   (page background)
  --color-white:         #FFFFFF
  --color-gray-100:      #F5F7FA
  --color-gray-200:      #E4E8EF
  --color-gray-400:      #9AA5B4
  --color-gray-600:      #4A5568
  --color-gray-800:      #1A202C
  --color-success:       #27AE60
  --color-success-bg:    #EAFAF1
  --color-warning:       #F39C12
  --color-warning-bg:    #FEF9E7
  --color-error:         #E74C3C
  --color-error-bg:      #FDEDEC
```

JOB STATUS COLOURS (match job state machine):
```
  --color-status-requested:      #9AA5B4  (gray)
  --color-status-pending:        #F39C12  (amber)
  --color-status-confirmed:      #3498DB  (blue)
  --color-status-in-progress:    #9B59B6  (purple)
  --color-status-complete:       #27AE60  (green)
  --color-status-disputed:       #E74C3C  (red)
  --color-status-released:       #27AE60  (green)
  --color-status-cancelled:      #95A5A6  (muted)
  --color-status-refunded:       #E67E22  (orange)
```

TYPOGRAPHY:
```
  --font-family:     'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif
  --font-size-xs:    11px
  --font-size-sm:    13px
```

```
  --font-size-base:  15px
  --font-size-md:    16px
  --font-size-lg:    18px
  --font-size-xl:    22px
  --font-size-2xl:   28px
  --font-size-3xl:   36px
  --font-weight-regular: 400
  --font-weight-medium:  500
  --font-weight-semibold: 600
  --font-weight-bold:    700
  --line-height-tight:   1.25
  --line-height-base:    1.5
  --line-height-relaxed: 1.75

SPACING (4px base unit):
  --space-1:   4px
  --space-2:   8px
  --space-3:   12px
  --space-4:   16px
  --space-5:   20px
  --space-6:   24px
  --space-8:   32px
  --space-10:  40px
  --space-12:  48px
  --space-16:  64px

SHAPE:
  --radius-sm:   4px
  --radius-md:   8px
  --radius-lg:   12px
  --radius-xl:   16px
  --radius-full: 9999px

SHADOW:
  --shadow-sm:  0 1px 3px rgba(0,0,0,0.08)
  --shadow-md:  0 4px 12px rgba(0,0,0,0.10)
  --shadow-lg:  0 8px 24px rgba(0,0,0,0.12)

Z-INDEX:
  --z-base:    0
  --z-above:   10
  --z-modal:   100
  --z-toast:   200

LAYOUT:
  --max-width-content: 1080px
```

```
  --header-height:     64px
  --sidebar-width:     240px
  --bottom-nav-height: 64px

2. globals.css:
  - Import tokens.css (using @import)
  - CSS reset: *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
  - Base body: font-family var(--font-family), font-size var(--font-size-base),
    line-height var(--line-height-base), color var(--color-gray-800),
    background var(--color-snow), -webkit-font-smoothing antialiased
  - Links: color inherit, text-decoration none
  - Images: max-width 100%, display block
  - Button: all unset, cursor pointer (CSS reset for buttons)
  - Input, textarea, select: font inherit
  - Helper utility classes:
    .sr-only — visually hidden but accessible to screen readers
    .container — max-width var(--max-width-content), margin 0 auto, padding 0 var(--space-6)
    .truncate — text overflow ellipsis, overflow hidden, white-space nowrap

3. Update index.html to add in <head>:
   <link rel="preconnect" href="https://fonts.googleapis.com">
   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
   <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap" rel="sty

Provide complete file contents for all three files.
```

P0-03 — Shared Components

Phase: 0 — UI Prototype | **Week:** W1 | **Depends on:** P0-02

Dependencies

- P0-02 complete (design tokens available)

Tools & Libraries

Tool / Library	Version	Purpose
React	18.x	Component framework
CSS Modules	Native (Vite)	Scoped component styles

Assumptions

- All components are purely presentational in Phase 0 — no API calls

- Props use default values rather than PropTypes validation
- CSS Modules files import design tokens via var(--token-name)
- Modal uses ReactDOM.createPortal to render into document.body
- StatusPill maps all 11 job state machine states to colours defined in tokens.css

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build all shared components for the SnowReach UI prototype (P0-03).

Create the following files in frontend/src/components/:

```
── Button ──────────────────────────────────────────────────────────────────
Button.jsx + Button.module.css

Props:
  variant: 'primary' | 'secondary' | 'ghost' | 'danger'  (default: 'primary')
  size: 'sm' | 'md' | 'lg'  (default: 'md')
  loading: boolean  (shows spinner, disables click)
  disabled: boolean
  fullWidth: boolean
  onClick: function
  type: 'button' | 'submit' | 'reset'  (default: 'button')
  children: node

Styles:
  - primary: background var(--color-primary), white text, hover darkens 10%
  - secondary: white background, primary border, primary text
  - ghost: transparent background, gray-600 text, hover gray-100 background
  - danger: background var(--color-error), white text
  - sm: height 32px, font-size sm, padding 0 space-3
  - md: height 40px, font-size base, padding 0 space-5
  - lg: height 48px, font-size md, padding 0 space-6
  - disabled/loading: opacity 0.5, cursor not-allowed
  - loading: show inline Spinner component, hide children text

── Input ───────────────────────────────────────────────────────────────────
Input.jsx + Input.module.css

Props:
  label: string
  error: string  (shows below input in red)
  hint: string  (shows below input in gray)
  type: string  (default: 'text')
```

```
  required: boolean
  disabled: boolean
  placeholder: string
  value, onChange, onBlur — pass through to <input>
  id: string (auto-generated from label if not provided)

Styles:
  - Label above input, font-size sm, font-weight medium, color gray-600
  - Input: full width, height 40px, border 1px solid gray-200, radius-md,
    padding 0 space-4, font-size base; focus: outline none, border-color primary,
    box-shadow 0 0 0 3px var(--color-primary-light)
  - Error state: border-color error, error message below in error colour
  - Hint: gray-400, font-size sm

── Card ──────────────────────────────────────────────────────────────────
Card.jsx + Card.module.css

Props:
  padding: 'sm' | 'md' | 'lg'  (default: 'md')
  shadow: boolean  (default: true)
  header: node  (optional, rendered above a divider)
  footer: node  (optional, rendered below a divider)
  children: node
  onClick: function  (if provided, card is clickable — hover effect)

Styles:
  - Background white, radius-lg
  - sm: padding space-4; md: space-6; lg: space-8
  - shadow-sm when shadow=true
  - clickable: hover translateY(-1px), shadow-md, cursor pointer, transition

── Modal ─────────────────────────────────────────────────────────────────
Modal.jsx + Modal.module.css

Props:
  isOpen: boolean
  onClose: function
  title: string
  size: 'sm' | 'md' | 'lg'  (default: 'md')
  children: node
  footer: node  (optional — renders below body with top border)

Behaviour:
  - Render via ReactDOM.createPortal into document.body
  - Click backdrop to close (call onClose)
  - Press Escape to close (useEffect + keydown listener)
```

- Trap focus inside modal while open
- Animate in: backdrop fade in, modal slide up slightly

Styles:
- Backdrop: fixed inset-0, background rgba(0,0,0,0.45), z-index var(--z-modal)
- Modal panel: centered (flex), white background, radius-lg, shadow-lg
- sm: max-width 400px; md: 560px; lg: 720px
- Header: title font-size lg font-weight semibold, close button top-right (×)
- Body: padding space-6, overflow-y auto, max-height 70vh

── Badge ──
Badge.jsx (no CSS module — styles inline via tokens)

Props:
variant: 'default' | 'primary' | 'success' | 'warning' | 'error' (default: 'default')
children: node

Styles (all inline using token values):
Pill shape, font-size xs, font-weight semibold, padding 2px 8px, radius-full
default: gray-200 bg, gray-600 text
primary: primary-light bg, primary-dark text
success: success-bg, success text
warning: warning-bg, warning text
error: error-bg, error text

── StatusPill ───
StatusPill.jsx (maps job state machine statuses to display)

Props:
status: one of the 11 job states (string)

Map each status to a { label, colorVar } using the --color-status-* tokens:
REQUESTED → 'Requested', --color-status-requested
PENDING_DEPOSIT → 'Awaiting Payment', --color-status-pending
CONFIRMED → 'Confirmed', --color-status-confirmed
IN_PROGRESS → 'In Progress', --color-status-in-progress
COMPLETE → 'Complete', --color-status-complete
INCOMPLETE → 'Incomplete', --color-status-warning
DISPUTED → 'Disputed', --color-status-disputed
RELEASED → 'Released', --color-status-released
REFUNDED → 'Refunded', --color-status-refunded
SETTLED → 'Settled', --color-status-complete
CANCELLED → 'Cancelled', --color-status-cancelled

Style: same pill shape as Badge, background is colorVar at 15% opacity,
text is colorVar. Use inline styles.

```
── Spinner ──────────────────────────────────────────────────────────────
Spinner.jsx

Props:
  size: 'sm' | 'md' | 'lg'  (default: 'md')
  color: string  (CSS colour, default: 'var(--color-primary)')

Pure CSS spinner: a div with border, border-top-color transparent, border-radius 50%,
animation spin 0.7s linear infinite.
sm: 16px, md: 24px, lg: 40px.
Include @keyframes spin in a <style> tag or inline via CSS injection.

── Index exports ────────────────────────────────────────────────────────
Create frontend/src/components/index.js that re-exports all components:
  export { default as Button } from './Button/Button'
  export { default as Input } from './Input/Input'
  ... etc.

Provide complete file contents for all files.
Run npm run lint and fix all errors before finishing.
```

P0-04 — Navigation Shell

Phase: 0 — UI Prototype | **Week:** W2 | **Depends on:** P0-03

Dependencies

- P0-03 complete (shared components available)
- P0-01 complete (React Router routes defined)

Tools & Libraries

Tool / Library	Version	Purpose
React Router	6.x	<Outlet />, <NavLink>
CSS Modules	Native	Layout styles

Assumptions

- Three distinct layouts: Requester (top nav + bottom nav on mobile), Worker (same), Admin (sidebar)

- A dev-only role switcher overlays the UI to switch between roles during prototype review
- Active nav link styling uses React Router's NavLink with isActive class
- The SnowReach logo is the SVG snowflake from assets/logo.svg
- No authentication logic yet — role is read from MockStateContext
- Bottom navigation is visible only on screens ≤ 768px

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Build the navigation shell layouts for the SnowReach prototype (P0-04).

Create or update these files:
  frontend/src/layouts/RequesterLayout.jsx + RequesterLayout.module.css
  frontend/src/layouts/WorkerLayout.jsx + WorkerLayout.module.css
  frontend/src/layouts/AdminLayout.jsx + AdminLayout.module.css
  frontend/src/assets/logo.svg
  frontend/src/context/MockStateContext.jsx
  frontend/src/components/DevRoleSwitcher/DevRoleSwitcher.jsx

── logo.svg ──────────────────────────────────────────────────────────────
Create a simple SVG snowflake logo (36×36 viewBox):
- Circle background in var(--color-primary)
- 4 lines crossing at center (horizontal, vertical, and two diagonals)
- Small circle at center
- All lines/circles in white
Export as a React component AND as a raw SVG file.

── MockStateContext.jsx ──────────────────────────────────────────────────
Create a React context that provides:
  currentRole: 'REQUESTER' | 'WORKER' | 'ADMIN'  (default: 'REQUESTER')
  setRole: function
  mockJobs: array of mock job objects (5 jobs in various states)
  mockUser: object { displayName, email, uid }
  setMockJobStatus: function(jobId, newStatus)

Wrap the app in this context provider in App.jsx.

── DevRoleSwitcher ───────────────────────────────────────────────────────
A floating panel (bottom-right corner, only visible in development mode).
Shows current role label and three buttons: Requester | Worker | Admin.
Clicking switches the role in MockStateContext, which triggers React Router
to navigate to the appropriate base route.
Style: white card, shadow-lg, z-index above everything, semi-transparent.
Hide completely in production (check import.meta.env.DEV).
```

── RequesterLayout.jsx ──
Top header (height: var(--header-height)):
  Left: SnowReach logo + wordmark "SnowReach"
  Center (desktop): NavLinks: Home (/requester/), Post a Job (/requester/post-job),
    My Jobs (links to most recent job or /requester/)
  Right: notification bell icon (badge with count 0), avatar circle with initials

Mobile bottom nav (visible ≤768px, hidden >768px):
  4 tabs with icons (use emoji as placeholder): □ Home, □ Post Job, □ Jobs, □ Profile
  Active tab highlighted in primary colour

Main content area:
  Renders <Outlet />
  Padding: space-6 on desktop, space-4 on mobile
  Max-width: var(--max-width-content), centered

── WorkerLayout.jsx ───
Same structure as RequesterLayout but different nav links:
  Desktop: Dashboard (/worker/), Active Job (/worker/active-job),
    Earnings (/worker/), Profile
  Mobile bottom nav: □ Dashboard, □ Job Alerts, □ Earnings, □ Profile

── AdminLayout.jsx ──
Sidebar layout (sidebar fixed on left, content scrolls on right):
  Sidebar (width: var(--sidebar-width)):
    Logo + "Admin Panel" label at top
    Nav links: □ Dashboard (/admin/), □ All Jobs (/admin/?tab=jobs),
      □ Users (/admin/?tab=users), □ Disputes (/admin/?tab=disputes),
      □ Analytics (/admin/analytics)
    Bottom of sidebar: current admin user name + logout button (mock)
  Main content: renders <Outlet />, padding space-8

On mobile (≤768px): sidebar becomes a top drawer (hamburger menu button in header).

After building all layouts, update App.jsx / router.jsx so that:
  - The MockStateContext currentRole drives which layout is shown
  - / redirects based on currentRole (REQUESTER → /requester/, etc.)
  - DevRoleSwitcher is rendered inside App.jsx, outside the router layouts

Provide complete file contents for all files.

P0-05 — Requester: Home and Job Posting Flow

Phase: 0 — UI Prototype | **Week:** W2 | **Depends on:** P0-04

Dependencies

- P0-04 complete (RequesterLayout with Outlet working)
- P0-03 complete (Button, Input, Card, Modal components)
- MockStateContext providing mockJobs and setMockJobStatus

Tools & Libraries

Tool / Library	Version	Purpose
React Router	6.x	useNavigate, Link
CSS Modules	Native	Page-scoped styles

Assumptions

- All data is mock; no API calls in Phase 0
- Job posting is a 4-step wizard with client-side state only
- On submit, a new mock job is added to MockStateContext and user navigates to /requester/jobs/:jobId
- Address geocoding step is simulated with a 1-second fake delay showing “3 Workers found in your area”
- Service pricing: driveway $45, walkway $20, steps $10, salting $15

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the Requester Home page and Job Posting flow (P0-05).

Files:
  frontend/src/pages/requester/Home.jsx + Home.module.css
  frontend/src/pages/requester/PostJob.jsx + PostJob.module.css

HOME PAGE:
Two sections:
1. Hero/CTA (when no active jobs): headline "Snow cleared. Fast.", subtext, large primary Button "Post a Job" -> /requester/post-job.
2. My Jobs list: map over mockJobs from MockStateContext. Each job as a Card: address, StatusPill, scheduled date, service types, "View Details" link. Empty state with icon and "Post your first job" CTA.

POST JOB PAGE (4-step wizard):
State: currentStep (1-4), formData, isSearchingWorkers.
Step indicator: 4 numbered circles, current step highlighted blue.

Step 1 - Location:
  Input: service address. Select: property type (House, Condo, Commercial).
  Select: driveway size (Small/Medium/Large).
  On Next: 1s delay, green banner "3 Workers available in your area", advance.

Step 2 - Services:
  Checkboxes: Driveway ($45), Walkway ($20), Steps ($10), Salting ($15).
  Running total shown: "Estimated: $XX + HST"

Step 3 - Schedule:
  Radio: ASAP | Specific Date & Time (date picker + 30-min time select 6am-9pm).
  Optional notes textarea (500 chars max, counter shown).

Step 4 - Review & Submit:
  Summary card. Price breakdown: Services, HST (13%), Platform fee (15%), Worker receives.
  Required acknowledgement checkbox.
  "Post Job" button: adds to MockStateContext with REQUESTED status,
  navigates to /requester/jobs/{newJobId}.

Validate before advancing. Provide complete file contents.

P0-06 — Requester: Job Status Tracking

Phase: 0 — UI Prototype | **Week:** W2 | **Depends on:** P0-05

Dependencies

- P0-05 complete, P0-03 complete, MockStateContext with setMockJobStatus

Tools & Libraries

Tool / Library	Version	Purpose
React Router	6.x	useParams to get jobId
CSS Modules	Native	Page styles

Assumptions

- Job data read from MockStateContext by jobId URL param

- Dev-only “Advance State” button cycles through job states for demo
- Cancel shown only for REQUESTED/PENDING_DEPOSIT/CONFIRMED; Dispute shown only for COMPLETE

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the Requester Job Status page (P0-06).

File: frontend/src/pages/requester/JobStatus.jsx + JobStatus.module.css

Read jobId from useParams(). Look up job in MockStateContext.

LAYOUT:
1. Header: "Job #SR-{jobId}" + StatusPill right-aligned.
2. Status timeline (horizontal desktop, vertical mobile):
   Requested -> Awaiting Payment -> Confirmed -> In Progress -> Complete.
   Current: blue filled circle. Past: green check. Future: gray empty.
3. Worker Card (CONFIRMED or later): initials avatar, "Alex M.", 4.8 stars,
   "1.2 km away", "Husqvarna ST224", "View Profile" link (opens WorkerProfile modal).
4. Job Details Card: address, services, scheduled time, notes, price breakdown.
5. Action Buttons:
   "Cancel Job" (red, if REQUESTED/PENDING_DEPOSIT/CONFIRMED):
     Confirm Modal -> setMockJobStatus CANCELLED.
   "Raise Dispute" (amber, if COMPLETE):
     Modal with reason textarea -> setMockJobStatus DISPUTED.
   "Rate Worker" (green, if COMPLETE):
     -> /requester/jobs/:jobId/rate
6. Dev tool (DEV only): floating "Advance State ->" cycles through all states.

Provide complete file contents.

P0-07 — Requester: Worker Profile Modal

Phase: 0 — UI Prototype | **Week:** W2 | **Depends on:** P0-03, P0-04

Dependencies

- P0-03 complete (Modal, Badge, Card)

Assumptions

- Dual-mode: standalone page at /requester/workers/:workerId AND modal overlay
- All Worker data hardcoded mock; 3 mock reviews

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the Requester Worker Profile view (P0-07).

File: frontend/src/pages/requester/WorkerProfile.jsx + WorkerProfile.module.css
Also export: WorkerProfileModal (wraps in Modal component)

CONTENT (hardcoded mock):
1. Header: avatar "AM" (primary blue circle), name "Alex M." (partially masked), rating 4.8 stars, "47 jobs completed. Member since Jan 2024", trust badges row: "Check Verified" "Insured" (shown as Phase 3 coming-soon).
2. Equipment: Husqvarna ST224, salt spreader, LED work light.
3. Service Area: "East York, Scarborough, Danforth". Response time < 30 min. Radius 5km.
4. Availability: Mon-Fri 6am-8pm, Sat-Sun 7am-6pm.
5. Reviews (3 mock): stars, date, text, reviewer first name.
6. Stats bar: Response rate 98%, Avg. job time 45 min, 0 disputes.

Standalone: "Back" link. Modal: title "Worker Profile".
Provide complete file contents.

P0-08 — Requester: Rating and Review Form

Phase: 0 — UI Prototype | **Week:** W3 | **Depends on:** P0-06, P0-03

Dependencies

- P0-06 complete (navigate from JobStatus), P0-03 complete

Assumptions

- Mutual rating: Requester rates Worker; Worker mock rating of Requester pre-filled
- On submit, setMockJobStatus called with RELEASED; confirmation screen shown
- Interactive star widget (click to select)

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the mutual Rating and Review form (P0-08).

File: frontend/src/pages/requester/RateWorker.jsx + RateWorker.module.css

STATE: rating (0-5), reviewText, wouldHireAgain (null), submitted (false)

BEFORE SUBMIT:
1. Header: "Rate Your Experience" + job address.
2. Rate Worker section:
   - "How did Alex M. do?"
   - Interactive 5-star widget: click selects, hover previews, amber colour.
   - Textarea (0-500 chars, counter). Would you hire again: "Yes/No" toggle buttons.
3. Worker rating of you (read-only, mock): 5 stars + positive comment.
4. "Submit Rating" button. Error if submitted with rating=0.

AFTER SUBMIT confirmation:
  Green check icon, "Thanks for your feedback!",
  "Your payment will be released within 2-3 business days.",
  payout breakdown.
  Buttons: "View Job Summary" | "Post Another Job"

Provide complete file contents.

P0-09 — Worker: Registration and Profile Setup

Phase: 0 — UI Prototype | **Week:** W3 | **Depends on:** P0-04, P0-03

Dependencies

- P0-04 complete (WorkerLayout), P0-03 complete

Assumptions

- 4-step wizard; client-side only, no API calls
- Photo upload is mock (preview via URL.createObjectURL, no actual upload)
- Stripe Connect is a mock button that simulates 2s delay then shows success
- Age 18+ enforced client-side

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the Worker registration and profile setup flow (P0-09).

File: frontend/src/pages/worker/Register.jsx + Register.module.css

4-step wizard with step indicator (same pattern as PostJob).

Step 1 - Personal Information:
  Inputs: full legal name, phone (Canadian format), date of birth (date picker).
  Validate 18+ years old.
  Required checkboxes: independent contractor acknowledgement + liability.
  Required checkbox: supervisor/adult present disclosure.

Step 2 - Equipment:
  Snowblower make/model (text), type (select), clearing width (select).
  Extras checkboxes: salt spreader, snow shovel, LED light, ice scraper.
  Mock photo upload (image/*, shows thumbnail preview).
  Years of experience (number 0-50).

Step 3 - Service Area & Availability:
  Postal code (Canadian M1A 1A1 format).
  Radius slider 1-15km. Below: "Approx. X properties in range" (radius^2 * 12).
  Availability grid: day checkboxes Mon-Sun, start/end time per day.
  Max jobs per day (1-10).

Step 4 - Payment Setup:
  Explanation card: payout mechanics, timeline, HST, platform fee.
  "Connect with Stripe" button: 2s loading -> "Bank account connected (Demo mode)".
  "Complete Registration" -> /worker/ with welcome banner.

Provide complete file contents.

P0-10 — Worker: Incoming Job Request Screen

Phase: 0 — UI Prototype | **Week:** W3 | **Depends on:** P0-04, P0-03

Dependencies

- P0-04 complete, P0-03 complete, MockStateContext

Assumptions

- Real 10-minute countdown (useEffect + setInterval)
- Address partially masked until accepted; auto-declines at timer=0

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the Worker incoming job request screen (P0-10).

File: frontend/src/pages/worker/JobRequest.jsx + JobRequest.module.css

STATE: timeRemaining (600 sec), decision (null | accepted | declined)
COUNTDOWN: setInterval -1/sec. At 0: auto-decline. Clear on unmount/decision set.
Flash red when < 2 minutes remain.

PENDING layout:
1. Amber header bar: "New Job Request - Respond within MM:SS"
2. Job summary card: partial address "1xx Maple St., East York", services,
   schedule ASAP, duration 45-60 min, special notes.
3. Earnings card: gross, HST kept, platform fee deducted, net payout, distance.
4. Large buttons: green "Accept Job" | red "Decline"
   Accept: decision=accepted, setMockJobStatus CONFIRMED.
   Decline: decision=declined.

ACCEPTED: green banner, full address revealed, requester contact,
  "Go to Active Job" -> /worker/active-job.

DECLINED/EXPIRED: gray card, "Request Passed" message, "Back to Dashboard" -> /worker/

Provide complete file contents.

P0-11 — Worker: Active Job Screen

Phase: 0 — UI Prototype | **Week:** W3 | **Depends on:** P0-10, P0-03

Dependencies

- P0-10 complete (CONFIRMED state), P0-03 complete, MockStateContext

Assumptions

- Photo upload is mock (client-side preview, no actual upload)
- Check In: CONFIRMED -> IN_PROGRESS; Mark Complete: IN_PROGRESS -> COMPLETE
- Map is a placeholder grey box

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the Worker Active Job execution screen (P0-11).

File: frontend/src/pages/worker/ActiveJob.jsx + ActiveJob.module.css

STATE: jobPhase (arriving | working | done), photos (File[], max 5)

LAYOUT:
1. Status banner: blue (arriving) | purple (working) | green (done).
2. Property card: full address, gray map placeholder (300px tall), special notes in amber box, job reference number.
3. Services checklist: checkboxes per service, progress "X/Y completed".
4. Photo documentation: file input (image/*, multiple), thumbnail grid, max 5 photos (disable input after 5), tip about dispute protection.
5. Timeline actions:
   If arriving: "I've Arrived - Start Job" button.
   On click: jobPhase=working, setMockJobStatus IN_PROGRESS.
   If working: "Mark Job Complete" button.
   Disabled until 1+ photo added. On click: jobPhase=done, setMockJobStatus COMPLETE.
6. Completion screen: green check, confirmation text, payout amount, "Back to Dashboard" button.

Provide complete file contents.

P0-12 — Worker: Earnings Dashboard

Phase: 0 — UI Prototype | **Week:** W3 | **Depends on:** P0-04, P0-03

Dependencies

- P0-04 complete (WorkerLayout), P0-03 complete

Assumptions

- All data hardcoded; this is the Worker home route (/worker/ index)
- 5 mock payout rows; HST shown as separate line item; Stripe link disabled/mock

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the Worker Earnings Dashboard (P0-12).

File: frontend/src/pages/worker/Earnings.jsx + Earnings.module.css
(This is also the Worker home screen at route /worker/)

SECTIONS:
1. Welcome header: "Welcome back, Alex" + Active StatusPill.
2. Pending payout card (primary border): "$156.30 pending", 3 jobs breakdown, release condition note, progress bar 2/3 jobs rated.
3. This Month stats (3 cards in row): Jobs 11, Gross $847.60, Net $720.46.
4. Payout history table: Date | Job Address | Services | Gross | Fee(15%) | HST | Net. 5 mock rows Jan-Mar 2026. HST tooltip about CRA remittance. Mobile: overflow-x scroll.
5. All-time stats: Total Jobs 47, Member Jan 2024, Rating 4.8 stars, On-time 96%.
6. Stripe section: "Connected - TD Bank ending 4521", disabled "Manage Payout Settings".

Provide complete file contents.

P0-13 — Admin: Dashboard

Phase: 0 — UI Prototype | **Week:** W3 | **Depends on:** P0-04, P0-03

Dependencies

- P0-04 complete (AdminLayout with sidebar), P0-03 complete

Assumptions

- All data hardcoded; jobs table links to /admin/jobs/:jobId
- Tabs (Jobs/Users/Disputes) implemented as client-side state

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the Admin Dashboard (P0-13).

File: frontend/src/pages/admin/Dashboard.jsx + Dashboard.module.css

SECTIONS:
1. Header: "Admin Dashboard" + today's date.
2. Four summary stat cards (2x2 on mobile, 4-in-row on desktop):
   Total Jobs Today: 23 (up 8%), Active Jobs Now: 7,
   Revenue Today: $1,847.00, Open Disputes: 2.
   Each with coloured left border: blue, purple, green, red.
3. Tabbed content (Recent Jobs | User Registrations | Open Disputes):
   Recent Jobs: 10 mock rows, columns Job ID/Requester/Worker/Status/Services/Value/Time.
     Row click -> /admin/jobs/:jobId
   User Registrations: 8 mock rows, columns Name/Role badge/Email/Registered/Status.
   Open Disputes: 2 mock rows, columns Job ID/Requester/Worker/Opened/Reason/"Review" link.
4. Recent Activity Feed (right sidebar desktop, below tables mobile):
   Timestamped events: completions, new registrations, dispute opened, payment released.

Provide complete file contents.

P0-14 — Admin: Job Detail and Dispute Resolution

Phase: 0 — UI Prototype | **Week:** W3 | **Depends on:** P0-13, P0-03

Dependencies

- P0-13 complete (navigation from Dashboard), P0-03 complete (Modal, Button, Card)

Assumptions

- All data mock (hardcoded job SR-039, status DISPUTED)
- Status override and all admin actions use confirmation Modals before applying

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the Admin Job Detail and Dispute Resolution page (P0-14).

File: frontend/src/pages/admin/JobDetail.jsx + JobDetail.module.css

Read jobId from useParams(). Use hardcoded mock job SR-039 (DISPUTED).

SECTIONS:
1. Header: "Back to Dashboard" link, Job ID + StatusPill, Requester -> Worker + date.
2. Timeline card: vertical list of state transitions with timestamps and actors:
   REQUESTED, PENDING_DEPOSIT, CONFIRMED, IN_PROGRESS, COMPLETE, DISPUTED.
3. Parties card (two columns):
   Requester: name, email, phone, account status.
   Worker: same + rating, background check "Verified", jobs completed.
4. Payment card: Gross/HST/Platform fee/Worker net, PaymentIntent ID, Escrow: HELD.
5. Dispute section (when DISPUTED):
   Requester statement, Worker statement, 2 mock evidence photo thumbnails.
   Resolution form: radio Release|Refund|Split, split % slider, notes textarea,
   "Resolve Dispute" -> confirm Modal -> apply mock.
6. Admin Actions card:
   Override status dropdown + "Apply Override" -> confirm Modal.
   "Issue Refund" -> confirm Modal -> success state "$88.27 refunded".
   "Release Payment" -> confirm Modal -> success state.
   Admin notes textarea + "Save Note".

Provide complete file contents.

P0-15 — Mobile Responsive Polish

Phase: 0 — UI Prototype | **Week:** W3 | **Depends on:** P0-05 through P0-14

Dependencies

- All P0-04 through P0-14 tasks complete

Assumptions

- Mobile-first breakpoints: 375px, 768px, 1024px, 1440px
- No horizontal scroll on any page at 375px except explicit overflow-x tables
- Modals use bottom-sheet pattern on mobile

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Audit and fix mobile responsiveness for all SnowReach prototype screens (P0-15).

Create: frontend/src/styles/responsive.css
Update globals.css to import responsive.css.
Update any component/page CSS module with issues.

REQUIRED FIXES:

1. Navigation:
   RequesterLayout/WorkerLayout: hide desktop nav (<=767px), show fixed bottom nav.
   Add padding-bottom = bottom-nav-height to main content on mobile.
   AdminLayout: sidebar hidden on mobile; hamburger button opens drawer overlay.

2. Grids: stat cards (P0-12, P0-13) -> 2-col mobile, 4-col desktop.
   Multi-column layouts stack vertically on mobile.

3. Tables: wrap in div overflow-x:auto. Gradient fade on right edge as scroll hint.

4. Forms: inputs full-width on mobile. Service checkboxes: 1 col mobile.

5. Modals: on mobile (<=767px): 100% viewport width, slide up from bottom (bottom sheet), max-height 90vh. Desktop: centered overlay.

6. Typography: reduce heading font sizes on mobile.

7. Action buttons in P0-06: stack vertically full-width on mobile.

8. P0-10 Accept/Decline: fixed to bottom of screen on mobile.

Provide all changed/created file contents.

P0-16 — Stakeholder Review Materials

Phase: 0 — UI Prototype | **Week:** W4 | **Depends on:** P0-15

Dependencies

- P0-15 complete (all screens polished and mobile-responsive)
- Prototype deployed to Firebase Hosting preview channel

Assumptions

- Produces two documents and deployment instructions; no application code
- Feedback from this session is triaged into: P1 scope / post-MVP / rejected

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Produce stakeholder review materials for the SnowReach UI prototype (P0-16).

Create two Markdown documents:

1. docs/stakeholder-review-template.md
   Sections:
   - Reviewer info: name, role, date
   - First impressions (3 questions, rated 1-5 + comments)
   - Requester flow feedback (4 items, rated 1-5): posting ease, pricing clarity, job tracking confidence, rating experience satisfaction
   - Worker flow feedback (3 items): registration clarity, job request ease, earnings clarity
   - Admin flow feedback (2 items): dashboard usefulness, dispute resolution clarity
   - Missing features (open list)
   - Confusing elements (open list)
   - Priority ranking: top 3 improvements before launch
   - Overall NPS (0-10): "How likely are you to use SnowReach?"

2. docs/prototype-demo-script.md
   Presenter script with timing:
   - Intro (2 min): what SnowReach is, prototype disclaimer
   - Requester demo (8 min): Home -> Post Job (4 steps) -> Status -> Rate
     Talking points: ease, transparent pricing, safety features
   - Worker demo (6 min): Register -> Job Request (show timer) -> Active Job -> Earnings
     Talking points: earnings transparency, documentation, payout timeline
   - Admin demo (4 min): Dashboard -> Job Detail -> Dispute Resolution
     Talking points: oversight, adjudication, analytics roadmap
   - Q&A prompts (5 min): suggested proactive questions

   Include deployment code block:
   ```
   cd snowreach/frontend && npm run build
   cd ../firebase && firebase hosting:channel:deploy prototype --expires 7d
   ```

Provide complete content for both Markdown files.

P1-01 — Firebase Project Setup

Phase: 1 — MVP | **Week:** W4 | **Depends on:** ENV-01 through ENV-05

Dependencies

- Firebase CLI installed and authenticated (ENV-01)
- Project folder structure established (ENV-05)
- Google account with GCP billing enabled

Tools & Libraries

Tool / Library	Version	Purpose
Firebase CLI	Latest	Project init, deploy
firebase-admin (Java SDK)	9.x	Backend Firestore/Auth/Storage access
Firebase JS SDK	10.x	React client-side Auth + Firestore reads

Assumptions

- Two Firebase projects created: snowreach-dev and snowreach-prod
- A separate Firebase project snowreach-audit hosts the audit log Firestore
- Service account JSON files are NEVER committed to Git (.gitignore covers them)
- Firebase Emulator Suite used for all local development from Phase 1 onward

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Set up Firebase projects and produce all configuration files (P1-01).

This is a setup + configuration task. Produce the following files:

1. firebase/firebase.json (final version replacing ENV-05 skeleton):

```
{
  "hosting": {
    "public": "../frontend/dist",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [{ "source": "**", "destination": "/index.html" }],
    "headers": [
      { "source": "/assets/**", "headers": [{ "key": "Cache-Control", "value": "public,max-age=31536000,immutab
    ]
  },
  "firestore": { "rules": "firestore.rules", "indexes": "firestore.indexes.json" },
  "storage": { "rules": "storage.rules" },
  "emulators": {
    "auth": { "port": 9099 },
    "firestore": { "port": 8080 },
    "storage": { "port": 9199 },
    "hosting": { "port": 5000 },
```

```
    "ui": { "enabled": true, "port": 4000 }
  }
}
```

2. firebase/.firebaserc:

```
{
  "projects": {
    "default": "snowreach-dev",
    "prod": "snowreach-prod"
  }
}
```

3. frontend/src/services/firebase.js:
Initialize Firebase app using environment variables. Export:
- app (FirebaseApp)
- auth (getAuth)
- db (getFirestore)
- storage (getStorage)

Use import.meta.env.VITE_ prefixed variables:
  VITE_FIREBASE_API_KEY, VITE_FIREBASE_AUTH_DOMAIN, VITE_FIREBASE_PROJECT_ID,
  VITE_FIREBASE_STORAGE_BUCKET, VITE_FIREBASE_MESSAGING_SENDER_ID, VITE_FIREBASE_
Also connect to emulators if import.meta.env.VITE_USE_EMULATORS === 'true'.

4. frontend/.env.example (template, safe to commit):

```
VITE_FIREBASE_API_KEY=your-api-key
VITE_FIREBASE_AUTH_DOMAIN=your-project.firebaseapp.com
VITE_FIREBASE_PROJECT_ID=your-project-id
VITE_FIREBASE_STORAGE_BUCKET=your-project.appspot.com
VITE_FIREBASE_MESSAGING_SENDER_ID=your-sender-id
VITE_FIREBASE_APP_ID=your-app-id
VITE_API_BASE_URL=http://localhost:8080
VITE_USE_EMULATORS=true
```

5. List of environment variables needed for backend (to be added to application-dev.yml):
FIREBASE_PROJECT_ID, FIREBASE_SERVICE_ACCOUNT_PATH,
FIREBASE_AUDIT_PROJECT_ID, FIREBASE_AUDIT_SERVICE_ACCOUNT_PATH

6. Firestore collection structure documentation comment block to add at top of
  firebase/firestore.rules — list all 13 collection paths:
  /users/{uid}, /workers/{uid}, /jobs/{jobId}, /jobRequests/{id},
  /ratings/{id}, /disputes/{disputeId}, /notifications/{uid}/feed/{notifId},
  /geocache/{hash}, /auditLog/{entryId} (in separate project),
  /analytics/daily/{date}, /analytics/summary/current,
  /fraudFlags/{flagId}, /adminReviewQueue/{id}

7. Step-by-step instructions (as comments in the files or a setup note) for:

- Running Firebase Emulator Suite: firebase emulators:start
- Deploying to dev: firebase use default && firebase deploy
- Deploying to prod: firebase use prod && firebase deploy

Provide complete file contents for all files listed.

P1-02 — GCP + Cloud Run + CI/CD

Phase: 1 — MVP | **Week:** W4 | **Depends on:** P1-01, ENV-01

Dependencies

- P1-01 complete (Firebase projects created)
- Google Cloud project created with billing enabled
- gcloud CLI authenticated (ENV-01)

Tools & Libraries

Tool / Library	Version	Purpose
Docker	Latest	Container build
Google Cloud Run	N/A	Backend hosting
Google Container Registry	N/A	Docker image storage
GitHub Actions	N/A	CI/CD pipeline

Assumptions

- Cloud Run is configured with min-instances=1 in prod (to avoid cold starts)
- Docker multi-stage build: Maven build stage + slim JRE 21 runtime stage
- CI/CD triggers on push to main branch; dev deployments are manual
- Backend environment variables are stored in GCP Secret Manager (not Cloud Run env var UI)

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Create the Dockerfile, CI/CD workflows, and Cloud Run configuration (P1-02).

Files to create:
  backend/Dockerfile
  .github/workflows/backend-deploy.yml
  .github/workflows/frontend-deploy.yml

1. backend/Dockerfile (multi-stage):

```
Stage 1 (builder): FROM maven:3.9-eclipse-temurin-21 AS builder
  WORKDIR /app
  COPY pom.xml .
  RUN mvn dependency:go-offline -B
  COPY src ./src
  RUN mvn package -DskipTests -B
Stage 2 (runtime): FROM eclipse-temurin:21-jre-jammy
  WORKDIR /app
  COPY --from=builder /app/target/*.jar app.jar
  EXPOSE 8080
  ENTRYPOINT ["java", "-jar", "app.jar"]
```

2. .github/workflows/backend-deploy.yml:

```
name: Backend Deploy
on:
  push:
    branches: [main]
    paths: [backend/**]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v2
        with:
          credentials_json: ${{ secrets.GOOGLE_CLOUD_SERVICE_ACCOUNT }}
      - uses: google-github-actions/setup-gcloud@v2
      - name: Build and push Docker image
        run: |
          gcloud builds submit backend/ --tag gcr.io/${{ secrets.GOOGLE_CLOUD_PROJECT }}/snowreach-api:
      - name: Deploy to Cloud Run
        run: |
          gcloud run deploy snowreach-api            --image gcr.io/${{ secrets.GOOGLE_CLOUD_PROJECT }}/sr
```

3. .github/workflows/frontend-deploy.yml:

```
name: Frontend Deploy
on:
  push:
    branches: [main]
    paths: [frontend/**]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
```

```
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
        cache: npm
        cache-dependency-path: frontend/package-lock.json
    - run: npm ci
      working-directory: frontend
    - run: npm run build
      working-directory: frontend
      env:
        VITE_FIREBASE_API_KEY: ${{ secrets.VITE_FIREBASE_API_KEY }}
        VITE_FIREBASE_AUTH_DOMAIN: ${{ secrets.VITE_FIREBASE_AUTH_DOMAIN }}
        VITE_FIREBASE_PROJECT_ID: ${{ secrets.VITE_FIREBASE_PROJECT_ID }}
        VITE_FIREBASE_STORAGE_BUCKET: ${{ secrets.VITE_FIREBASE_STORAGE_BUCKET }}
        VITE_FIREBASE_MESSAGING_SENDER_ID: ${{ secrets.VITE_FIREBASE_MESSAGING_SENDER
        VITE_FIREBASE_APP_ID: ${{ secrets.VITE_FIREBASE_APP_ID }}
        VITE_API_BASE_URL: ${{ secrets.VITE_API_BASE_URL }}
    - uses: w9jds/firebase-action@master
      with:
        args: deploy --only hosting
      env:
        FIREBASE_TOKEN: ${{ secrets.FIREBASE_TOKEN }}
```

Provide complete file contents. Also list all GitHub Secrets that must be set and which workflow/step uses each one.

P1-03 — Spring Boot Skeleton

Phase: 1 — MVP | **Week:** W4 | **Depends on:** ENV-01 (Maven, Java 21 installed)

Dependencies

- Java 21 and Maven installed (ENV-01)
- Backend directory structure created (ENV-05)

Tools & Libraries

Tool / Library	Version	Purpose
Spring Boot	3.2.x	Application framework
spring-boot-starter-web	3.2.x	REST API
spring-boot-starter-security	3.2.x	Security filter chain

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firebase Admin SDK
quartz	2.3.x	Job scheduling
stripe-java	25.x	Stripe payments
sendgrid-java	4.x	Email delivery
google-maps-services	2.x	Geocoding
h2	2.x	In-memory DB for Quartz
spring-boot-starter-test	3.2.x	Testing
jackson-databind	(via Boot)	JSON

Assumptions

- All secrets loaded from environment variables; no hardcoded values anywhere
- pom.xml is the single source of truth for all dependency versions
- application.yml uses Spring profiles: dev (H2 Quartz, Firebase Emulator) and prod (Cloud Run)
- Global exception handler returns RFC 7807 Problem JSON

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Create the Spring Boot project skeleton (P1-03).

Files to create/update:
  backend/pom.xml
  backend/src/main/resources/application.yml
  backend/src/main/resources/application-dev.yml
  backend/src/main/resources/application-prod.yml
  backend/src/main/java/com/snowreach/SnowReachApplication.java
  backend/src/main/java/com/snowreach/config/CorsConfig.java
  backend/src/main/java/com/snowreach/exception/GlobalExceptionHandler.java
  backend/src/main/java/com/snowreach/exception/JobNotFoundException.java
  backend/src/main/java/com/snowreach/exception/InvalidTransitionException.java
  backend/src/main/java/com/snowreach/exception/PaymentException.java

1. pom.xml:
   Parent: spring-boot-starter-parent 3.2.3
   Java version: 21
   Dependencies:
     spring-boot-starter-web
     spring-boot-starter-security
     firebase-admin 9.3.0
```

quartz 2.3.2
spring-context-support (for Quartz integration)
stripe-java 25.3.0
sendgrid-java 4.10.1
google-maps-services 2.2.0
com.h2database h2 (scope: runtime)
spring-boot-starter-actuator
spring-boot-starter-test (scope: test)
Build plugin: spring-boot-maven-plugin

2. application.yml (base, all profiles inherit):
spring.application.name: snowreach-api
spring.profiles.active: ${SPRING_PROFILE:dev}
server.port: 8080
management.endpoints.web.exposure.include: health
management.endpoint.health.show-details: always
snowreach:
firebase.project-id: ${FIREBASE_PROJECT_ID}
firebase.service-account-path: ${FIREBASE_SERVICE_ACCOUNT_PATH:}
firebase.audit-project-id: ${FIREBASE_AUDIT_PROJECT_ID:}
stripe.secret-key: ${STRIPE_SECRET_KEY}
stripe.webhook-secret: ${STRIPE_WEBHOOK_SECRET}
sendgrid.api-key: ${SENDGRID_API_KEY}
sendgrid.from-email: noreply@snowreach.ca
sendgrid.from-name: SnowReach
maps.api-key: ${MAPS_API_KEY}

3. application-dev.yml:
snowreach.firebase.use-emulator: true
snowreach.firebase.emulator-host: localhost:8080
spring.quartz.job-store-type: memory
logging.level.com.snowreach: DEBUG
logging.level.org.springframework.security: DEBUG

4. application-prod.yml:
snowreach.firebase.use-emulator: false
spring.quartz.job-store-type: memory
logging.level.com.snowreach: INFO

5. CorsConfig.java:
@Configuration that registers a CorsFilter allowing:
- Origins: http://localhost:3000, https://snowreach-dev.web.app, https://snowreach.ca
(load allowed origins from snowreach.cors.allowed-origins config property)
- Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS
- Headers: Authorization, Content-Type, Accept
- Allow credentials: true

- Max age: 3600

6. GlobalExceptionHandler.java:
   @RestControllerAdvice with handlers returning ResponseEntity<Map<String,Object>>
   in RFC 7807 Problem JSON format (fields: type, title, status, detail, instance).
   Handle:
   - JobNotFoundException -> 404
   - InvalidTransitionException -> 409 Conflict
   - PaymentException -> 402 Payment Required
   - MethodArgumentNotValidException -> 400 with field error details
   - AccessDeniedException -> 403
   - Exception (catch-all) -> 500 (log stack trace, do NOT expose it in response)

7. Exception classes:
   JobNotFoundException extends RuntimeException (String jobId constructor)
   InvalidTransitionException extends RuntimeException (String message)
   PaymentException extends RuntimeException (String message, Throwable cause)

8. SnowReachApplication.java: standard @SpringBootApplication main class.

Provide complete file contents for all files.

P1-04 — Firebase Auth Integration and RBAC

Phase: 1 — MVP | **Week:** W5 | **Depends on:** P1-03, P1-01

Dependencies

- P1-03 complete (Spring Boot skeleton with security starter)
- P1-01 complete (Firebase project exists, service account JSON available)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	verifyIdToken(), custom claims
spring-boot-starter-security	3.2.x	SecurityFilterChain

Assumptions

- Firebase ID token is passed in every request as Authorization: Bearer {token}
- Token verification happens on every request (no session/cookie)

- Roles are stored in Firestore users/{uid}.roles[] and mirrored to Firebase custom claims
- /api/health, /webhooks/** are excluded from auth
- The RBAC annotation @RequiresRole("ADMIN") is implemented via a HandlerInterceptor

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Implement Firebase Auth token verification and RBAC (P1-04).

Files to create/update:
  backend/src/main/java/com/snowreach/config/FirebaseConfig.java
  backend/src/main/java/com/snowreach/config/SecurityConfig.java
  backend/src/main/java/com/snowreach/security/FirebaseTokenFilter.java
  backend/src/main/java/com/snowreach/security/RbacInterceptor.java
  backend/src/main/java/com/snowreach/security/RequiresRole.java  (annotation)
  backend/src/main/java/com/snowreach/security/AuthenticatedUser.java  (record/POJO)
  backend/src/main/java/com/snowreach/config/WebMvcConfig.java

1. FirebaseConfig.java:
   @Configuration that initializes FirebaseApp on startup.
   Read FIREBASE_SERVICE_ACCOUNT_PATH from config. If path is non-empty, load from file.
   If path is empty (emulator mode, local dev without service account), use
   FirebaseOptions.Builder with a mock credential or the emulator default.
   Also initialize a second FirebaseApp named "audit" for the audit log Firestore project.
   Expose FirebaseAuth bean and two Firestore beans: primary and audit.

2. FirebaseTokenFilter.java (extends OncePerRequestFilter):
   - Extract token from Authorization header (strip "Bearer " prefix)
   - Call FirebaseAuth.getInstance().verifyIdToken(token)
   - On success: create AuthenticatedUser(uid, email, roles[]) and set on
     SecurityContextHolder as UsernamePasswordAuthenticationToken
   - On FirebaseAuthException: return 401 JSON error
   - On missing/malformed header for protected routes: return 401
   - Skip filter for /api/health and /webhooks/**

3. SecurityConfig.java:
   @Configuration @EnableWebSecurity
   Configure HttpSecurity:
   - stateless session (SessionCreationPolicy.STATELESS)
   - disable CSRF (REST API with Bearer tokens)
   - permit: /api/health, /webhooks/**, /actuator/health
   - authenticate all other requests
   - Add FirebaseTokenFilter before UsernamePasswordAuthenticationFilter
```

4. RequiresRole.java:
   Custom annotation: @Target(METHOD) @Retention(RUNTIME) @interface RequiresRole { String[] value()

5. RbacInterceptor.java (implements HandlerInterceptor):
   In preHandle: check if handler method has @RequiresRole annotation.
   If yes: get AuthenticatedUser from SecurityContext, check if any role matches.
   If no match: throw AccessDeniedException("Insufficient role").
   If annotation absent: pass through.

6. AuthenticatedUser.java:
   Record or POJO: uid (String), email (String), roles (List<String>)
   Helper: hasRole(String role) -> boolean

7. WebMvcConfig.java: @Configuration implementing WebMvcConfigurer, adds RbacInterceptor.

Provide complete file contents for all files.

P1-05 — User Registration and Profile API

Phase: 1 — MVP | **Week:** W5 | **Depends on:** P1-04

Dependencies

- P1-04 complete (auth filter and RBAC working)
- P1-03 complete (pom.xml with firebase-admin dependency)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firestore write, custom claims
spring-boot-starter-validation	3.2.x	Input validation annotations

Assumptions

- User creates their Firebase Auth account on the client (email/password or OAuth) before calling POST /api/users
- POST /api/users is called once after signup to create the Firestore user document and set the role claim
- Minimum age 18 validated on date of birth

- Email uniqueness enforced by Firebase Auth (not duplicated in Firestore check)

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement the User registration and profile API (P1-05).

Files to create/update:
  backend/src/main/java/com/snowreach/controller/UserController.java
  backend/src/main/java/com/snowreach/service/UserService.java
  backend/src/main/java/com/snowreach/model/User.java
  backend/src/main/java/com/snowreach/dto/CreateUserRequest.java

Firestore schema for users/{uid}:
  uid: String, email: String, displayName: String, phone: String,
  dateOfBirth: String (ISO-8601 date), roles: List<String> (["REQUESTER"] or ["WORKER"]),
  status: String (ACTIVE | SUSPENDED | BANNED), fcmToken: String,
  createdAt: Timestamp, updatedAt: Timestamp

1. CreateUserRequest.java (Jakarta validation annotations):
   @NotBlank displayName, @Email @NotBlank email,
   @Pattern(regexp="^[0-9]{3}-[0-9]{3}-[0-9]{4}$") phone,
   @NotNull dateOfBirth (LocalDate), @NotNull role (String: REQUESTER or WORKER)
   Custom validation: dateOfBirth must be >= 18 years before today.

2. User.java (POJO matching Firestore schema):
   All fields with getters/setters or use Lombok @Data if available,
   otherwise plain Java. toMap() method returning Map<String,Object> for Firestore write.

3. UserService.java:
   createUser(String uid, CreateUserRequest req) -> User:
     - Validate 18+ age
     - Build user document, set status=ACTIVE, createdAt=now
     - Call AuditLogService.write(...) BEFORE Firestore write (null before, user after)
     - Write to Firestore users/{uid}
     - Set Firebase custom claim: { "roles": [req.getRole()] } via FirebaseAuth
     - Return User
   getUser(String uid) -> User: read from Firestore, throw JobNotFoundException if absent.
   updateUser(String uid, Map<String,Object> updates) -> User:
     - Only allow updating: displayName, phone, fcmToken
     - Set updatedAt=now
     - Audit log the change
     - Merge update into Firestore document

```
4. UserController.java:
  POST /api/users (auth required, any authenticated user):
    - Get uid from AuthenticatedUser in SecurityContext
    - Call userService.createUser(uid, request)
    - Return 201 Created with User body
  GET /api/users/{uid} (auth required, own uid or ADMIN):
    - If not own uid and not ADMIN: 403
    - Return User
  PATCH /api/users/{uid} (auth required, own uid or ADMIN):
    - Return updated User
```

Provide complete file contents.

P1-06 — Worker Profile API

Phase: 1 — MVP | **Week:** W5 | **Depends on:** P1-05

Dependencies

- P1-05 complete (user registration working; uid exists in Firestore)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firestore read/write
spring-boot-starter-validation	3.2.x	Input validation

Assumptions

- Worker profile is a separate Firestore document at workers/{uid}
- A user must have role=WORKER to have/create a worker profile
- Service area is stored as centroid GeoPoint + radius km (not a polygon in Phase 1)
- stripeAccountId is set later by PaymentService when Stripe Connect onboarding completes

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement the Worker Profile API (P1-06).

Files:
backend/src/main/java/com/snowreach/controller/WorkerController.java
backend/src/main/java/com/snowreach/service/WorkerService.java
backend/src/main/java/com/snowreach/model/Worker.java
backend/src/main/java/com/snowreach/dto/WorkerProfileRequest.java

Firestore schema for workers/{uid}:
uid, displayName, equipmentList (List<String>), equipmentPhotoUrl,
serviceAreaCentroid (Map with lat/lng doubles), serviceRadiusKm (double),
availabilitySchedule (Map<String, Map<String,String>> dayOfWeek -> {start,end}),
maxJobsPerDay (int), maxConcurrentJobs (int, default 1),
averageRating (double, default 0.0), totalJobsCompleted (int, default 0),
stripeAccountId (String), backgroundCheckStatus (String, default PENDING),
insuranceStatus (String, default NONE),
isActive (boolean, default false -- activated when background check passes in Phase 3,
set to true manually for Phase 1 testing),
createdAt, updatedAt

1. WorkerProfileRequest.java:
equipmentList (List<String>, not empty), serviceAreaCentroid (required, lat/lng),
serviceRadiusKm (1.0-25.0), availabilitySchedule (Map), maxJobsPerDay (1-10)

2. WorkerService.java:
createOrUpdateProfile(String uid, WorkerProfileRequest req) -> Worker
getWorker(String uid) -> Worker
setActive(String uid, boolean active) -> void (admin action)
updateAverageRating(String uid, double newRating) -> void (called by RatingService)
getActiveWorkers() -> List<Worker> (used by MatchingService)

3. WorkerController.java:
POST /api/workers (WORKER role, creates own profile)
GET /api/workers/{uid} (any auth user -- used when Requester views Worker)
PATCH /api/workers/{uid} (own uid or ADMIN)
GET /api/workers (ADMIN only, paginated, optional status filter)
PATCH /api/workers/{uid}/active (ADMIN only -- set isActive)
POST /api/workers/{uid}/connect-onboard (WORKER own uid -- calls PaymentService for Stripe link)

Provide complete file contents.

P1-07 — Geocoding Service

Phase: 1 — MVP | **Week:** W5 | **Depends on:** P1-03

Dependencies

- P1-03 complete (application.yml with maps.api-key config)
- Google Maps API key with Geocoding API enabled in GCP Console

Tools & Libraries

Tool / Library	Version	Purpose
google-maps-services	2.2.x	Server-side geocoding
firebase-admin	9.3.x	Firestore cache

Assumptions

- Google Maps API key is NEVER sent to the browser; all geocoding is server-side only
- Geocode results are cached in Firestore geocache/{hash} for 30 days
- The FSA (Forward Sortation Area) fallback uses a hardcoded map of 30+ GTA/Ontario FSAs
- Cache key = SHA-256 of the normalized address string (lowercase, stripped whitespace)

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Implement the server-side Geocoding service with FSA fallback and Firestore cache (P1-07).

Files:
  backend/src/main/java/com/snowreach/service/GeocodingService.java
  backend/src/main/java/com/snowreach/util/GeoUtils.java

1. GeoUtils.java:
   haversineDistanceKm(double lat1, double lon1, double lat2, double lon2) -> double
   normalizeAddress(String address) -> String (lowercase, collapse whitespace, trim)
   extractFSA(String postalCode) -> String (first 3 chars uppercase)

2. GeocodingService.java:
   Primary method: geocode(String address) -> double[] {lat, lng}

   Algorithm:
   a. Normalize address, compute cache key = SHA-256 of normalized string
   b. Check Firestore geocache/{cacheKey}: if exists and cachedAt < 30 days ago, return cached
   c. Call Google Maps GeocodingApi.geocode(context, address).await()
      - Use GeoApiContext built with maps.api-key from config
```

d. If result is empty or status != OK: fall back to FSA lookup
e. FSA lookup: extract postal code from address if present; look up in FSA_CENTROIDS map
f. If FSA not found: throw IllegalArgumentException("Could not geocode address: " + address)
g. On success: write to geocache/{cacheKey} with coordinates and cachedAt timestamp
h. Return double[] {lat, lng}

FSA_CENTROIDS: private static final Map<String, double[]> containing at minimum:
M1A -> Toronto East, M4J -> East York, M5V -> Toronto Downtown, M6H -> Dufferin,
M2N -> North York, M9A -> Etobicoke, L3R -> Markham, L6P -> Brampton,
L4W -> Mississauga, K1A -> Ottawa, L2G -> Niagara Falls, N2J -> Waterloo,
N6A -> London, L8P -> Hamilton, P3E -> Sudbury, T2P -> Calgary, V5K -> Vancouver
(Include at least 20 entries covering major Ontario areas)

Note: NEVER log or return the raw API key. Catch Google API exceptions and
fall back to FSA rather than propagating them as 500 errors.

Provide complete file contents.

P1-08 — Job Posting API

Phase: 1 — MVP | **Week:** W6 | **Depends on:** P1-05, P1-07

Dependencies

- P1-05 complete (user profile exists)
- P1-07 complete (GeocodingService available)
- P1-04 complete (auth working)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firestore atomic writes

Assumptions

- Job IDs are UUID v4 generated by the backend (not Firestore auto-ID)
- A Requester may only have one non-terminal job at a time (REQUESTED/PENDING_DEPOSIT/CONFIRMED/IN_PROGRESS)
- Price calculation: all amounts stored as integer cents CAD
- The job is created in REQUESTED state; transitions to PENDING_DEPOSIT after Stripe PaymentIntent created (P1-11)
- Matching and dispatch kick off asynchronously after job creation (P1-09, P1-10)

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement the Job posting API (P1-08).

Files:
  backend/src/main/java/com/snowreach/controller/JobController.java
  backend/src/main/java/com/snowreach/service/JobService.java (initial version)
  backend/src/main/java/com/snowreach/model/Job.java
  backend/src/main/java/com/snowreach/dto/CreateJobRequest.java

Firestore schema for jobs/{jobId}:
  jobId (String UUID), requesterId, status, serviceTypes (List<String>),
  address (String), coordinates (Map with lat/lng),
  scheduledTime (Timestamp, null means ASAP), estimatedDurationMin (int),
  specialNotes (String), matchedWorkerIds (List<String>),
  currentWorkerId (String), currentWorkerRequestId (String),
  stripePaymentIntentId (String), depositAmountCents (int),
  platformFeeCents (int), hstCents (int), netWorkerCents (int),
  photoUrls (List<String>),
  createdAt, updatedAt, confirmedAt, startedAt, completedAt, cancelledAt, disputedAt

1. CreateJobRequest.java:
   @NotBlank address, @NotEmpty serviceTypes (List<String>),
   scheduledTime (Instant, optional -- null = ASAP),
   @Size(max=500) specialNotes

2. Job.java: POJO matching schema. toMap() method for Firestore.
   Status enum or String constants: REQUESTED, PENDING_DEPOSIT, CONFIRMED,
   IN_PROGRESS, COMPLETE, INCOMPLETE, DISPUTED, RELEASED, REFUNDED, SETTLED, CANCE

3. JobService.java createJob method:
   a. Validate: requester exists, no open non-terminal job for this requester
   b. Geocode address via GeocodingService
   c. Calculate amounts:
      - base price: sum of service prices (driveway=4500, walkway=2000, steps=1000, salting=1500 cents)
      - hst = round(base * 0.13)
      - platform fee = round(base * 0.15)
      - net worker = base - platform_fee + hst (HST flows through to Worker)
      - deposit amount = base + hst (Requester pays base + HST)
   d. Generate jobId = UUID.randomUUID().toString()
   e. Build Job object, status=REQUESTED
   f. Call AuditLogService.write("system", "JOB_CREATED", "job", jobId, null, job)
   g. Write to Firestore jobs/{jobId}
   h. Kick off matching asynchronously: @Async annotated call to MatchingService

i. Return created Job

4. JobController.java:
   POST /api/jobs (REQUESTER role): returns 201 with Job body
   GET /api/jobs/{jobId} (requester or assigned worker or ADMIN)
   GET /api/jobs (ADMIN only, paginated, filter by status/requesterId/workerId)
   POST /api/jobs/{jobId}/cancel: delegates to cancelJob in JobService (P1-14)

Provide complete file contents.

P1-09 — Worker Matching Algorithm

Phase: 1 — MVP | **Week:** W6 | **Depends on:** P1-06, P1-07, P1-08

Dependencies

- P1-06 complete (Worker profiles in Firestore)
- P1-07 complete (GeoUtils.haversineDistanceKm available)
- P1-08 complete (Job model available)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firestore query
spring-scheduling	3.2.x	@Async support

Assumptions

- In Phase 1, background check filter is skipped (all active Workers eligible)
- A Worker is “busy” if they have any job in CONFIRMED or IN_PROGRESS state with them as currentWorkerId
- Sorting: rating DESC, then distance ASC
- Maximum 20 Workers in matchedWorkerIds list

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement the Worker matching algorithm (P1-09).

File: backend/src/main/java/com/snowreach/service/MatchingService.java

@Service class with method:
  matchAndStoreWorkers(String jobId) -> List<String> (ordered Worker uid list)

Algorithm:
1. Read job from Firestore to get coordinates and serviceTypes.
2. Query workers collection: where isActive == true.
3. For each Worker:
   a. Compute distance using GeoUtils.haversineDistanceKm(job coords, worker centroid).
   b. Skip if distance > worker.serviceRadiusKm.
   c. Check if Worker is currently busy: query jobs where currentWorkerId == workerUid and status in [CONFIRMED, IN_PROGRESS] -- if any result, skip this Worker.
   d. Also check if Worker has reached maxConcurrentJobs (Phase 2 feature; in Phase 1 maxConcurrentJobs is always 1, so "busy" check above is sufficient).
4. Sort remaining Workers: averageRating DESC, then distance ASC.
5. Take first 20.
6. Extract uid list.
7. Update job document: set matchedWorkerIds = uid list, updatedAt = now.
8. Return uid list.

Also annotate with @Async so it does not block the POST /api/jobs response.
Enable @EnableAsync in a config class or SnowReachApplication.

Provide complete file contents including the @EnableAsync addition.

P1-10 — Sequential Dispatch with Quartz

Phase: 1 — MVP | **Week:** W6 | **Depends on:** P1-09, P1-03

Dependencies

- P1-09 complete (matchedWorkerIds stored on job)
- P1-03 complete (Quartz dependency in pom.xml, QuartzConfig placeholder)

Tools & Libraries

Tool / Library	Version	Purpose
Quartz Scheduler	2.3.x	Durable 10-minute dispatch timer
H2	2.x	In-memory Quartz job store (Phase 1)
firebase-admin	9.3.x	Read/write Firestore jobRequests

Assumptions

- Quartz is H2 in-memory in Phase 1; state is lost on restart
- On startup, DispatchService checks for any PENDING jobRequests in Firestore and reschedules Quartz timers
- jobRequests/{jobId}_{workerId} document tracks the current dispatch attempt
- NotificationService (P1-18) is called to push the request to the Worker — stub call in P1-10, wired in P1-18

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Implement sequential Worker dispatch with Quartz timer (P1-10).

Files:
  backend/src/main/java/com/snowreach/config/QuartzConfig.java
  backend/src/main/java/com/snowreach/service/DispatchService.java
  backend/src/main/java/com/snowreach/scheduler/DispatchJob.java

Firestore schema for jobRequests/{jobId}_{workerId}:
  jobId, workerId, status (PENDING | ACCEPTED | DECLINED | EXPIRED),
  sentAt (Timestamp), expiresAt (Timestamp), respondedAt (Timestamp)

1. QuartzConfig.java:
   @Configuration that configures Quartz with H2 in-memory store.
   Properties: org.quartz.jobStore.class = RAMJobStore,
   org.quartz.threadPool.threadCount = 5.
   Expose SchedulerFactoryBean as a Spring bean.
   On startup (implement ApplicationListener<ContextRefreshedEvent>):
     Call DispatchService.recoverPendingDispatches() to reschedule any jobs
     that were in-flight when the server last stopped.

2. DispatchService.java:
   dispatchToNextWorker(String jobId):
     a. Read job from Firestore, get matchedWorkerIds and currentWorkerRequestId.
     b. Determine next Worker uid in the list (the one after any already-tried ones).
        Track tried Workers by reading all jobRequest docs for this jobId.
     c. If no untried Workers remain: set job status=CANCELLED, notify Requester.
     d. Create jobRequests/{jobId}_{workerId} doc with status=PENDING, expiresAt=now+10min.
     e. Call notificationService.sendJobRequest(workerId, jobId) -- stub OK for now.
     f. Schedule Quartz DispatchJob to fire in 10 minutes with jobData: jobId, workerId.

   handleWorkerResponse(String jobId, String workerId, boolean accepted):
     a. Read jobRequest doc, validate it is still PENDING.
```

```
    b. If accepted: update jobRequest status=ACCEPTED, update job.currentWorkerId=workerId,
       transition job to CONFIRMED (call JobService.transition).
    c. If declined: update jobRequest status=DECLINED, call dispatchToNextWorker(jobId).

  recoverPendingDispatches():
    Query all jobRequest docs where status=PENDING and expiresAt > now.
    For each: compute remaining time, schedule Quartz job for that time.
    If expiresAt < now: immediately expire and dispatch next.

  Also expose an endpoint in JobController or a new WorkerController method:
  POST /api/job-requests/{requestId}/respond (WORKER role)
    Body: { accepted: boolean }
    Calls handleWorkerResponse.

3. DispatchJob.java (implements Job):
  In execute(JobExecutionContext): get jobId and workerId from JobDataMap.
  Read jobRequest from Firestore; if still PENDING: set status=EXPIRED,
  call DispatchService.dispatchToNextWorker(jobId).

Provide complete file contents.
```

P1-11 — Stripe Payment Intents (Escrow)

Phase: 1 — MVP | **Week:** W8 | **Depends on:** P1-08, P1-03

Dependencies

- P1-08 complete (Job created with deposit amounts)
- P1-03 complete (Stripe dependency in pom.xml)
- Stripe test mode API key available

Tools & Libraries

Tool / Library	Version	Purpose
stripe-java	25.x	Payment Intents API
spring-boot-starter-web	3.2.x	Webhook endpoint

Assumptions

- Stripe is used in test mode during development; live mode keys only in production

- PaymentIntent is created with capture_method=manual to hold funds in escrow
- The client confirms the PaymentIntent using Stripe.js (frontend); backend only creates it and listens to webhooks
- Idempotency key = jobId to prevent duplicate charges on retry
- Webhook events are idempotent: check Firestore before processing any event

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Implement Stripe Payment Intents for escrow (P1-11).

Files:
  backend/src/main/java/com/snowreach/service/PaymentService.java  (initial)
  backend/src/main/java/com/snowreach/controller/PaymentController.java  (initial)
  backend/src/main/java/com/snowreach/controller/WebhookController.java  (initial)

1. PaymentService.java:
   createPaymentIntent(String jobId) -> String clientSecret:
     a. Read job from Firestore, get depositAmountCents.
     b. Build PaymentIntentCreateParams:
        amount = depositAmountCents, currency = "cad",
        capture_method = MANUAL (hold in escrow),
        metadata = { jobId: jobId },
        idempotencyKey = "pi_create_" + jobId
     c. Call Stripe.paymentIntents.create() with idempotency key header.
     d. Store paymentIntentId on job document.
     e. Return clientSecret for frontend.

   confirmCapture(String paymentIntentId):
     Call PaymentIntent.capture() to finalize the hold.
     Called when job transitions to CONFIRMED.

2. PaymentController.java:
   POST /api/jobs/{jobId}/payment-intent (REQUESTER role, must own the job):
     Returns { clientSecret: "..." } for Stripe.js to confirm.

3. WebhookController.java:
   POST /webhooks/stripe (no auth -- verified by signature):
     a. Read raw request body as String.
     b. Get Stripe-Signature header.
     c. Verify: Webhook.constructEvent(payload, sigHeader, webhookSecret).
     d. Check event type:
        "payment_intent.succeeded":
```

- Extract jobId from event.data.object.metadata.
- Check Firestore stripeEvents/{eventId} -- if already processed, return 200.
- Write stripeEvents/{eventId} = { processedAt: now }.
- Call jobService.transition(jobId, "CONFIRMED", "stripe", null).

"payment_intent.payment_failed":
- Extract jobId.
- Notify Requester (stub call to notificationService).
- Do NOT auto-cancel job -- let Requester retry.

e. Return 200 OK for all events (even unhandled -- Stripe retries on non-200).

Provide complete file contents including all import statements.

P1-12 — Stripe Connect Express (Worker Payouts)

Phase: 1 — MVP | **Week:** W8 | **Depends on:** P1-11, P1-06

Dependencies

- P1-11 complete (PaymentService exists)
- P1-06 complete (Worker profile has stripeAccountId field)
- Stripe Connect enabled on the Stripe account (done in Stripe Dashboard)

Tools & Libraries

Tool / Library	Version	Purpose
stripe-java	25.x	Connect Express, Transfers

Assumptions

- Workers use Stripe Connect Express (not Standard or Custom)
- Platform account retains commission automatically via Stripe's transfer mechanism
- HST is included in the Transfer amount (Workers receive net + HST; they remit HST to CRA)
- Payout timing: Transfer is created 2 business days after RELEASED (via Quartz delay)

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement Stripe Connect Express for Worker payouts (P1-12).

Add to:
backend/src/main/java/com/snowreach/service/PaymentService.java
backend/src/main/java/com/snowreach/controller/PaymentController.java

1. createConnectOnboardingLink(String workerUid, String returnUrl, String refreshUrl) -> String:
   a. Read Worker doc. If stripeAccountId is null/empty:
      Create Account with type=express, country=CA, email from Worker.
      Store accountId in Firestore workers/{uid}.stripeAccountId.
   b. Create AccountLink with type=account_onboarding, return_url, refresh_url.
   c. Return the AccountLink url.

2. releasePayment(String jobId) -> void:
   a. Read job from Firestore: get netWorkerCents, hstCents, currentWorkerId.
   b. Read Worker: get stripeAccountId. If null: log error, do not release.
   c. Create Transfer:
      amount = netWorkerCents + hstCents,
      currency = "cad",
      destination = worker.stripeAccountId,
      transfer_group = jobId,
      metadata = { jobId, workerId: currentWorkerId }
      idempotency key = "transfer_" + jobId
   d. Update job: set stripeTransferId, status=RELEASED (via JobService.transition).
   e. Notify Worker of payout amount (stub notificationService call).
   Note: The 15% commission stays on the platform account automatically
   (Transfer amount is netWorkerCents, not gross).

3. Add to PaymentController:
   POST /api/workers/{uid}/connect-onboard (WORKER own uid):
     Returns { onboardingUrl: "..." }
   POST /api/jobs/{jobId}/release-payment (ADMIN role only):
     Calls releasePayment(jobId)

4. Add refund support to PaymentService:
   refundJob(String jobId) -> void:
     Read job.stripePaymentIntentId.
     Create Refund with payment_intent = intentId.
     Update job status to REFUNDED via JobService.transition.

Add refund endpoint to PaymentController:
   POST /api/jobs/{jobId}/refund (ADMIN only)

Provide complete file contents for all additions.

P1-13 — Job State Machine

Phase: 1 — MVP | **Week:** W9 | **Depends on:** P1-08, P1-04, P1-11

Dependencies

- P1-08 complete (JobService scaffolded)
- P1-04 complete (AuthenticatedUser available in SecurityContext)
- AuditLogService stub available (will be completed in P1-20)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firestore transactions

Assumptions

- All state transitions go through a single transition() method (no direct status field writes elsewhere)
- Firestore transactions ensure the “check current state + write new state” is atomic
- The transition table is hardcoded in a Map<String, Set> (fromState -> allowedToStates)
- Actor permission checks enforce who can trigger each transition

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement the complete job state machine in JobService (P1-13).

Update: backend/src/main/java/com/snowreach/service/JobService.java

Add the complete transition() method:
  transition(String jobId, String targetStatus, String actorUid, String reason) -> Job

VALID TRANSITIONS (fromState -> toStates):
  REQUESTED -> [PENDING_DEPOSIT, CANCELLED]
  PENDING_DEPOSIT -> [CONFIRMED, CANCELLED]
  CONFIRMED -> [IN_PROGRESS, CANCELLED]
  IN_PROGRESS -> [COMPLETE, INCOMPLETE]
  COMPLETE -> [DISPUTED, RELEASED] (RELEASED happens after ratings, DISPUTED within 2hr)
  INCOMPLETE -> [DISPUTED, RELEASED]
  DISPUTED -> [RELEASED, REFUNDED]

```
  RELEASED -> [SETTLED]
  REFUNDED -> []
  SETTLED -> []
  CANCELLED -> []

ACTOR PERMISSION GUARDS (enforce in transition()):
  REQUESTED -> PENDING_DEPOSIT: system (Stripe webhook)
  PENDING_DEPOSIT -> CONFIRMED: system (Stripe webhook)
  CONFIRMED -> IN_PROGRESS: currentWorkerId == actorUid
  IN_PROGRESS -> COMPLETE: currentWorkerId == actorUid
  IN_PROGRESS -> INCOMPLETE: currentWorkerId == actorUid OR ADMIN
  COMPLETE -> DISPUTED: requesterId == actorUid (within 2hr of completedAt)
  COMPLETE -> RELEASED: system (after ratings submitted)
  INCOMPLETE -> DISPUTED: requesterId == actorUid
  INCOMPLETE -> RELEASED: ADMIN
  DISPUTED -> RELEASED: ADMIN
  DISPUTED -> REFUNDED: ADMIN
  RELEASED -> SETTLED: system (after payout confirmed)
  * -> CANCELLED: requesterId OR ADMIN (subject to P1-14 cancellation rules)

IMPLEMENTATION:
  Use Firestore runTransaction() to:
  1. Read current job status inside transaction.
  2. Validate transition is allowed (throw InvalidTransitionException if not).
  3. Validate actor permission (throw AccessDeniedException if not).
  4. Build update map: status, updatedAt, and relevant timestamp field
     (confirmedAt, startedAt, completedAt, cancelledAt, disputedAt as appropriate).
  5. BEFORE writing: call AuditLogService.write("transition", actorUid, "job", jobId, before, after).
     (AuditLogService is injected; use a stub if P1-20 not yet complete.)
  6. Write update inside transaction.
  7. Return updated Job.

Also add SIDE EFFECTS after transaction (not inside, to avoid transaction bloat):
  On -> CONFIRMED: call paymentService.confirmCapture(job.stripePaymentIntentId)
  On -> RELEASED: schedule Quartz job to call paymentService.releasePayment(jobId) in 0ms
    (immediate for Phase 1; can add delay later)
  On -> CANCELLED: call paymentService.refundJob(jobId) if payment was captured
  On any transition: call notificationService.notifyJobTransition(jobId, oldStatus, newStatus)

Provide complete updated JobService.java.
```

P1-14 — Cancellation and $10 Fee

Phase: 1 — MVP | **Week:** W9 | **Depends on:** P1-13, P1-11

Dependencies

- P1-13 complete (transition() method in JobService)
- P1-11 complete (PaymentService with Stripe PaymentIntent)

Tools & Libraries

Tool / Library	Version	Purpose
stripe-java	25.x	Partial capture + refund

Assumptions

- $10 CAD = 1000 cents cancellation fee
- Fee is charged only when cancelling from CONFIRMED state (worker already accepted)
- Stripe partial capture: capture $10 + HST ($11.30), refund the rest
- CANCELLED from REQUESTED or PENDING_DEPOSIT = full refund (no capture needed for REQUESTED since PaymentIntent not yet confirmed)

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement job cancellation with conditional $10 CAD fee (P1-14).

```
Update:
  backend/src/main/java/com/snowreach/service/JobService.java
  backend/src/main/java/com/snowreach/service/PaymentService.java
  backend/src/main/java/com/snowreach/controller/JobController.java

1. JobService.cancelJob(String jobId, String actorUid) -> Job:
   a. Read job. Validate actorUid is the requesterId OR has ADMIN role.
   b. Get current status:
      - REQUESTED: transition to CANCELLED, no charge (PaymentIntent not yet confirmed).
      - PENDING_DEPOSIT: cancel the PaymentIntent, transition to CANCELLED.
      - CONFIRMED: apply $10 fee -- call paymentService.cancelWithFee(jobId).
        Then transition to CANCELLED.
      - IN_PROGRESS or later: throw InvalidTransitionException
        ("Job cannot be cancelled once in progress. Use dispute process.").
   c. Return updated Job.

2. PaymentService.cancelWithFee(String jobId) -> void:
   a. Read job: stripePaymentIntentId, depositAmountCents.
```

b. CANCELLATION_FEE_CENTS = 1000 (10 CAD)
c. HST_ON_FEE = round(1000 * 0.13) = 130 cents
d. CHARGE_AMOUNT = 1130 cents
e. REFUND_AMOUNT = depositAmountCents - CHARGE_AMOUNT
f. Capture PaymentIntent with amount_to_capture = CHARGE_AMOUNT.
g. Create Refund for (depositAmountCents - CHARGE_AMOUNT) using the same PaymentIntent.
h. Update job: cancellationFeeCents = 1000, cancellationHstCents = 130.
i. Notify both Requester (fee charged) and Worker (job cancelled).

3. JobController additions:
POST /api/jobs/{jobId}/cancel (REQUESTER or ADMIN):
Calls jobService.cancelJob(jobId, actorUid). Returns updated Job.

Provide complete updated file sections.

P1-15 — Firebase Storage: Image Upload

Phase: 1 — MVP | **Week:** W11 | **Depends on:** P1-13, P1-01

Dependencies

- P1-01 complete (Firebase Storage configured)
- P1-13 complete (job state machine; photos allowed in IN_PROGRESS/COMPLETE only)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firebase Storage upload
spring-boot-starter-web	3.2.x	Multipart file handling

Assumptions

- Maximum 5 photos per job, 10 MB each
- Accepted formats: JPEG and PNG only (validated by MIME type check)
- Filenames are sanitized: stored as UUID + original extension, original name not preserved
- Download URLs are signed with 365-day expiry (for simplicity in Phase 1)

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement job photo upload via Firebase Storage (P1-15).

Files:
  backend/src/main/java/com/snowreach/controller/StorageController.java
  backend/src/main/java/com/snowreach/service/StorageService.java

1. StorageService.java:
   uploadJobPhoto(String jobId, MultipartFile file) -> String (download URL):
   a. Validate MIME type: only image/jpeg, image/png. Throw IllegalArgumentException otherwise.
   b. Validate size: file.getSize() <= 10 * 1024 * 1024. Throw if exceeded.
   c. Generate filename: UUID.randomUUID() + original extension (extract from content type).
   d. Storage path: "jobs/" + jobId + "/photos/" + filename
   e. Use Firebase Admin Storage SDK:
      Bucket bucket = StorageClient.getInstance().bucket();
      BlobId blobId = BlobId.of(bucket.getName(), storagePath);
      BlobInfo blobInfo = BlobInfo.newBuilder(blobId).setContentType(file.getContentType()).build();
      bucket.getStorage().create(blobInfo, file.getBytes());
   f. Generate signed URL (365-day expiry) or use public URL if bucket is public-read.
   g. Return download URL.

2. StorageController.java:
   POST /api/jobs/{jobId}/photos (WORKER role, must be currentWorkerId):
   a. Read job from Firestore. Validate:
      - actorUid == job.currentWorkerId
      - job.status in [IN_PROGRESS, COMPLETE] (photos allowed in both states)
      - job.photoUrls.size() < 5
   b. Call storageService.uploadJobPhoto(jobId, file).
   c. Atomically append URL to job.photoUrls in Firestore.
   d. Return { url: downloadUrl, totalPhotos: newCount }

Provide complete file contents.

P1-16 — Ratings and Reviews

Phase: 1 — MVP | **Week:** W11 | **Depends on:** P1-13, P1-06

Dependencies

- P1-13 complete (job must be in COMPLETE state for rating)
- P1-06 complete (WorkerService.updateAverageRating available)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firestore read/write

Assumptions

- One rating per party: requester rates worker, worker rates requester; stored as separate documents
- Rating document ID: {jobId}_requester and {jobId}_worker
- After BOTH ratings submitted, trigger job transition to RELEASED (which triggers payout)
- Worker's average rating is a rolling average updated on each new rating
- Requester ratings are stored but not currently displayed (future feature)

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Implement ratings and reviews (P1-16).

Files:
  backend/src/main/java/com/snowreach/controller/RatingController.java
  backend/src/main/java/com/snowreach/service/RatingService.java
  backend/src/main/java/com/snowreach/model/Rating.java
  backend/src/main/java/com/snowreach/dto/RatingRequest.java

Firestore schema for ratings/{jobId}_{raterRole}:
  jobId, raterUid, rateeUid, raterRole (REQUESTER | WORKER),
  stars (int 1-5), reviewText (String), wouldRepeat (boolean),
  createdAt (Timestamp)

1. RatingRequest.java:
   @Min(1) @Max(5) int stars, @Size(max=500) String reviewText, boolean wouldRepeat

2. RatingService.java:
   submitRating(String jobId, String raterUid, RatingRequest req) -> void:
   a. Read job. Validate status == COMPLETE.
   b. Determine raterRole: if raterUid == job.requesterId -> REQUESTER,
      else if raterUid == job.currentWorkerId -> WORKER, else throw 403.
   c. Check rating not already submitted: read ratings/{jobId}_{raterRole} -- throw if exists.
   d. Build Rating, write to Firestore ratings/{jobId}_{raterRole}.
   e. Audit log.
   f. If raterRole == WORKER: update Worker averageRating:
```

Read current Worker averageRating and totalJobsCompleted.
newAvg = (currentAvg * totalJobs + req.stars) / (totalJobs + 1)
Update Worker: averageRating = newAvg, totalJobsCompleted += 1.

g. Check if OTHER rating also submitted:
Read ratings/{jobId}_{otherRole}. If both exist:
Call jobService.transition(jobId, "RELEASED", "system", "Both ratings submitted").

3. RatingController.java:
POST /api/jobs/{jobId}/rating (REQUESTER or WORKER, must be party to job):
Calls ratingService.submitRating. Returns 201.
GET /api/jobs/{jobId}/ratings (job parties or ADMIN):
Returns both rating documents (or partial if only one submitted).

Provide complete file contents.

P1-17 — SendGrid Email Notifications

Phase: 1 — MVP | **Week:** W12 | **Depends on:** P1-05, P1-03

Dependencies

- P1-03 complete (sendgrid-java in pom.xml, API key in config)
- P1-05 complete (user email available via uid)

Tools & Libraries

Tool / Library	Version	Purpose
sendgrid-java	4.10.x	Email delivery

Assumptions

- Domain verification for the sender domain MUST be completed in SendGrid before launch (SPF, DKIM, DMARC DNS records)
- Email templates are inline HTML in Java strings (no external template files in Phase 1)
- All emails are sent asynchronously (@Async) so they don't block the request
- If SendGrid fails, log the error but do not fail the main operation

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement SendGrid email notification service (P1-17).

File: backend/src/main/java/com/snowreach/service/NotificationService.java (email section)

Create or update NotificationService.java with email sending capability.
Use SendGrid Java SDK: com.sendgrid.SendGrid, com.sendgrid.helpers.mail.Mail, etc.

Implement these 8 email methods (all @Async, all wrapped in try-catch logging errors):

1. sendWelcomeEmail(String toEmail, String displayName, String role)
   Subject: "Welcome to SnowReach, {name}!"
   HTML body: welcome message, role-specific getting started instructions,
   link to login.

2. sendJobConfirmedEmail(String requesterId, String workerId, Job job)
   Two emails: one to Requester, one to Worker.
   Requester: "Your job has been confirmed! Alex M. is on the way."
   Worker: "Job confirmed at {address}. You're up!"
   Include: address, scheduled time, price breakdown.

3. sendJobInProgressEmail(String requesterId, Job job)
   "Alex M. has arrived and started your job."

4. sendJobCompleteEmail(String requesterId, String workerId, Job job)
   Two emails. Include payout amounts for Worker email, reminder to rate for both.

5. sendPayoutReleasedEmail(String workerId, int amountCents, String jobId)
   "Your payment of ${amount} has been released and will arrive in 2-3 business days."

6. sendDisputeOpenedEmail(String requesterId, String workerId, String adminEmail, String jobId)
   Three emails. Notify parties dispute is under review. Notify Admin to adjudicate.

7. sendDisputeResolvedEmail(String requesterId, String workerId, String resolution, Job job)
   Two emails explaining the outcome (RELEASED or REFUNDED or SPLIT).

8. sendCancellationEmail(String requesterId, String workerId, boolean feeCharged, int feeCents)
   Two emails. Requester: job cancelled, fee amount if charged. Worker: job cancelled.

DOMAIN VERIFICATION NOTE:
Include a comment block at top of file:
// IMPORTANT: Before any emails will reach recipients (not spam), you must:
// 1. Go to SendGrid -> Settings -> Sender Authentication -> Authenticate Your Domain
// 2. Add the provided DNS records (SPF, DKIM) to your domain registrar
// 3. Add DMARC record: _dmarc.yourdomain.com TXT "v=DMARC1; p=none; rua=mailto:..."
// 4. Verify the domain in SendGrid before going to production

Provide complete file contents.

P1-18 — Firebase FCM Push Notifications

Phase: 1 — MVP | **Week:** W12 | **Depends on:** P1-17, P1-01

Dependencies

- P1-17 complete (NotificationService shell exists)
- P1-01 complete (Firebase Admin SDK initialized)
- P1-05 complete (fcmToken stored on user document)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	FirebaseMessaging.send()

Assumptions

- FCM token is stored on the user document (users/{uid}.fcmToken) and updated via PATCH /api/users/{uid}
- Every notification is also written to Firestore notifications/{uid}/feed/{notifId} for in-app feed
- The React client subscribes to the notifications feed collection for real-time updates
- If FCM send fails (invalid token), log the error but do not fail the operation

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement Firebase FCM push notifications (P1-18).

Update: backend/src/main/java/com/snowreach/service/NotificationService.java

Add push notification capability alongside the existing email methods.

1. Core sendPush method:
   sendPush(String uid, String notifType, String title, String body, Map<String,String> data):
   a. Read user.fcmToken from Firestore.
   b. If fcmToken is null or empty: skip FCM send (log debug).

```
  c. Build FCM Message:
     Message.builder()
       .setToken(fcmToken)
       .setNotification(Notification.builder().setTitle(title).setBody(body).build())
       .putAllData(data)
       .build()
  d. Call FirebaseMessaging.getInstance().send(message).
  e. Write to Firestore notifications/{uid}/feed/{UUID}:
     { notifId, type: notifType, title, body, data, isRead: false, createdAt: now }
  f. Catch FirebaseMessagingException: log warning, continue.

2. Implement these typed notification methods calling sendPush():
  notifyJobRequest(String workerUid, String jobId, String address, int payoutCents)
  notifyJobConfirmed(String requesterId, String workerName, String jobId)
  notifyWorkerArrived(String requesterId, String jobId)
  notifyJobComplete(String requesterId, String jobId)
  notifyJobComplete(String workerUid, String jobId, int payoutCents)
  notifyRatingRequest(String uid, String jobId)
  notifyPaymentReleased(String workerUid, int amountCents)
  notifyDisputeOpened(String uid, String jobId)
  notifyDisputeResolved(String uid, String jobId, String resolution)
  notifyCancellation(String uid, String jobId, boolean feeCharged)
  notifyJobTransition(String jobId, String oldStatus, String newStatus)
    (reads job, notifies relevant parties based on transition)

3. Add FCM token registration endpoint to UserController:
  PATCH /api/users/{uid}/fcm-token:
    Body: { fcmToken: "..." }
    Updates users/{uid}.fcmToken in Firestore.
```

Provide complete updated NotificationService.java and UserController addition.

P1-19 — Admin Dashboard Wired to Live Data

Phase: 1 — MVP | **Week:** W13 | **Depends on:** P1-08, P1-05, P1-06

Dependencies

- P1-08 complete (jobs collection populated)
- P1-05, P1-06 complete (users and workers collections populated)
- P0-13, P0-14 complete (Admin React pages exist with mock data)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firestore aggregation queries
Axios	1.x	Frontend HTTP client

Assumptions

- Summary stats are computed on-demand (no pre-aggregated analytics collection yet – that is P2-06)
- Firestore count() aggregate queries are used where available (Firestore SDK 9.4+)
- Frontend replaces all mockData references with real API calls using Axios

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Wire the Admin Dashboard to live backend data (P1-19).

Backend files:
  backend/src/main/java/com/snowreach/controller/AdminController.java

Frontend files to update:
  frontend/src/pages/admin/Dashboard.jsx
  frontend/src/pages/admin/JobDetail.jsx
  frontend/src/services/api.js

1. api.js (Axios base client):
   Create Axios instance with baseURL from import.meta.env.VITE_API_BASE_URL.
   Add request interceptor: attach Firebase ID token as Authorization: Bearer {token}.
   Get token using: getAuth().currentUser?.getIdToken().
   Add response interceptor: on 401, redirect to /login.
   Export named API methods:
     getAdminStats(), getAdminJobs(params), getJob(jobId),
     getAdminUsers(params), overrideJobStatus(jobId, status, reason),
     issueRefund(jobId), releasePayment(jobId)

2. AdminController.java:
   All endpoints require ADMIN role.
   GET /api/admin/stats:
     Returns: { jobsToday, activeJobs, revenueToday (cents), openDisputes,
       newUsersToday, totalWorkers, totalRequesters }
     Compute via Firestore queries (filter jobs by createdAt >= today start).
   GET /api/admin/jobs:
     Paginated (page, size params), filter by status, requesterId, workerId, date range.
```

Returns: { items: [Job], totalCount, page, size }
GET /api/admin/users:
Paginated, filter by role, status.
Returns: { items: [User], totalCount, page, size }
PATCH /api/admin/jobs/{jobId}/status:
Body: { targetStatus, reason }. Calls jobService.transition(). Audit logged.
POST /api/admin/jobs/{jobId}/refund: calls paymentService.refundJob().
POST /api/admin/jobs/{jobId}/release: calls paymentService.releasePayment().

3. Dashboard.jsx update:
Replace all mockData with useEffect calling api.getAdminStats() on mount.
Replace job table data with api.getAdminJobs() with loading/error states.
Add pagination controls (prev/next page buttons).

4. JobDetail.jsx update:
Replace hardcoded mock job with useEffect calling api.getJob(jobId).
Wire admin action buttons to api.overrideJobStatus(), api.issueRefund(), api.releasePayment().
Show success/error toasts after actions.

Provide complete file contents.

P1-20 — Audit Log Service

Phase: 1 — MVP | **Week:** W13 | **Depends on:** P1-01, P1-03

Dependencies

- P1-01 complete (separate Firestore audit project initialized as “audit” FirebaseApp)
- P1-03 complete (HashUtils placeholder created)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Write to separate Firestore project
java.security.MessageDigest	JDK 21	SHA-256 hashing

Assumptions

- The audit Firestore is in a completely separate Firebase project (snowreach-audit)
- The chain head (hash of genesis entry) is seeded as all-zeros string

- Daily integrity check runs at 2 AM via Quartz
- AuditLogService.write() is the ONLY way to write to the audit log; no direct Firestore access

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement the append-only Audit Log Service with SHA-256 hash chaining (P1-20).

```
Files:
  backend/src/main/java/com/snowreach/service/AuditLogService.java
  backend/src/main/java/com/snowreach/model/AuditEntry.java
  backend/src/main/java/com/snowreach/util/HashUtils.java
  backend/src/main/java/com/snowreach/scheduler/AuditIntegrityJob.java

Firestore collection (in AUDIT project): auditLog/{entryId}
  entryId (String UUID), timestamp (Timestamp), actorUid, action, entityType,
  entityId, before (Map), after (Map), hash (String), previousHash (String),
  sequenceNumber (long)

1. HashUtils.java:
   sha256(String input) -> String (hex-encoded)
   buildHashInput(String previousHash, long timestamp, String actorUid,
     String action, String entityId, String beforeJson, String afterJson) -> String
   (concatenate with | separator)

2. AuditLogService.java:
   Inject: Firestore auditFirestore (the "audit" FirebaseApp's Firestore instance)
   Inject: ObjectMapper (Jackson, for JSON serialization)

   write(String actorUid, String action, String entityType,
         String entityId, Object before, Object after) -> void:
   a. Convert before and after to JSON strings (null -> "null")
   b. Get current sequenceNumber: read auditLog/_meta/sequenceCounter document,
      increment atomically using Firestore transaction.
   c. Get previousHash: read auditLog/_meta/chainHead document (previousHash field).
      If not exists: previousHash = "0000000000000000000000000000000000000000000000000000000000000000
   d. Compute hash: sha256(buildHashInput(previousHash, now.epochMilli, actorUid, action,
        entityId, beforeJson, afterJson))
   e. Write auditLog/{UUID} with all fields.
   f. Update auditLog/_meta/chainHead: { previousHash: newHash, lastUpdated: now }
   g. Update auditLog/_meta/sequenceCounter: { value: newSeq }

3. AuditIntegrityJob.java (Quartz Job, runs at 2 AM daily):
   Read all auditLog documents from the previous day (ordered by sequenceNumber).
```

Re-compute each hash and verify it matches stored hash.
If any mismatch: log CRITICAL error and send alert email to admin.
Log summary: "Audit integrity check: X entries verified, 0 mismatches."

Provide complete file contents.

P1-21 — Firestore Security Rules

Phase: 1 — MVP | **Week:** W14 | **Depends on:** P1-08, P1-05, P1-06

Dependencies

- All Firestore collection schemas finalized (P1-05 through P1-18)
- Firebase Emulator Suite available for testing rules

Tools & Libraries

Tool / Library	Version	Purpose
Firebase Emulator Suite	Latest	Test security rules locally
firebase-admin	N/A	Rules do not apply to Admin SDK

Assumptions

- Firebase Admin SDK (backend) bypasses all security rules — only client-side reads are governed
- Client-side reads permitted: own user doc, own worker doc, own jobs, own notifications feed, own ratings
- Workers can read job request docs addressed to them
- Admins (identified by custom claim roles containing ADMIN) can read everything
- All client writes are denied for operational collections

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement complete Firestore and Storage security rules (P1-21).

Files:
firebase/firestore.rules
firebase/storage.rules

1. firestore.rules:
   Implement the following access rules:

```
Helper functions:
- isSignedIn(): request.auth != null
- isOwner(uid): isSignedIn() && request.auth.uid == uid
- isAdmin(): isSignedIn() && 'ADMIN' in request.auth.token.roles
- isWorker(): isSignedIn() && 'WORKER' in request.auth.token.roles
- isRequester(): isSignedIn() && 'REQUESTER' in request.auth.token.roles

Collection rules:
users/{uid}:
  read: isOwner(uid) || isAdmin()
  write: false  (all writes via Admin SDK)

workers/{uid}:
  read: isSignedIn()  (any authenticated user can read worker profiles)
  write: false

jobs/{jobId}:
  read: isSignedIn() && (
    resource.data.requesterId == request.auth.uid ||
    resource.data.currentWorkerId == request.auth.uid ||
    isAdmin()
  )
  write: false

jobRequests/{requestId}:
  read: isSignedIn() && (
    resource.data.workerId == request.auth.uid ||
    isAdmin()
  )
  write: false

ratings/{ratingId}:
  read: isSignedIn() && (
    resource.data.raterUid == request.auth.uid ||
    resource.data.rateeUid == request.auth.uid ||
    isAdmin()
  )
  write: false

notifications/{uid}/feed/{notifId}:
  read: isOwner(uid)
  write: false  (written by Admin SDK only)
  update: isOwner(uid) && request.resource.data.keys().hasOnly(['isRead'])
```

```
      (allow marking as read)

  disputes/{disputeId}:
    read: isSignedIn() && (
      resource.data.openedByUid == request.auth.uid ||
      isAdmin()
    )
    write: false

  // Deny all for: geocache, analytics, fraudFlags, adminReviewQueue, stripeEvents
  match /{document=**} {
    allow read, write: if false;
  }

2. storage.rules:
  Implement:
  jobs/{jobId}/photos/{fileName}:
    read: isSignedIn()  (any party to the job or admin -- full authorization done in backend)
    write: false  (all writes via Admin SDK)
  disputes/{disputeId}/{party}/{fileName}:
    read: isAdmin()
    write: false
  workers/{uid}/insurance/{fileName}:
    read: isAdmin() || isOwner(uid)
    write: false

Include comments explaining why client writes are denied for each collection.
Provide complete file contents.
```

P1-22 — Integration Test Suite

Phase: 1 — MVP | **Week:** W14 | **Depends on:** P1-13, P1-09, P1-16

Dependencies

- P1-13 complete (state machine implemented)
- P1-09 complete (matching logic implemented)
- P1-16 complete (rating logic)
- Firebase Emulator Suite configured in application-test.yml

Tools & Libraries

Tool / Library	Version	Purpose
spring-boot-starter-test	3.2.x	@SpringBootTest, MockMvc
Mockito	5.x (via Boot test)	Mock Stripe, SendGrid
firebase-admin	9.3.x	Emulator-connected Firestore
H2	2.x	In-memory Quartz store

Assumptions

- Tests run against Firebase Emulator (FIRESTORE_EMULATOR_HOST env var set)
- Stripe API calls are mocked with @MockBean
- SendGrid calls are mocked with @MockBean
- Each test method resets Firestore data via emulator REST API or setup/teardown helpers

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Write the integration test suite (P1-22).

Files:
  backend/src/test/java/com/snowreach/JobServiceTest.java
  backend/src/test/java/com/snowreach/MatchingServiceTest.java
  backend/src/test/java/com/snowreach/StateMachineTest.java
  backend/src/main/resources/application-test.yml

1. application-test.yml:
   spring.profiles: test
   snowreach.firebase.use-emulator: true
   snowreach.firebase.emulator-host: localhost:8080
   spring.quartz.job-store-type: memory
   (Mock API keys to prevent accidental real calls)

2. JobServiceTest.java:
   @SpringBootTest @ActiveProfiles("test")
   @MockBean PaymentService, NotificationService

   Test methods:
   - testCreateJob_success(): create valid job, verify Firestore doc, verify REQUESTED status
   - testCreateJob_duplicateOpenJob(): second job creation throws exception
   - testCreateJob_invalidAddress(): GeocodingService throws, verify 400 behavior
   - testCancelJob_fromRequested_noFee(): cancel from REQUESTED, verify no Stripe call
   - testCancelJob_fromConfirmed_feeCharged(): verify paymentService.cancelWithFee() called
```

- testCancelJob_fromInProgress_throws(): verify InvalidTransitionException

3. MatchingServiceTest.java:
   Test matching logic with test Worker and Job documents:
   - testMatchWorkers_filtersByRadius(): Worker outside radius is excluded
   - testMatchWorkers_filtersBusyWorkers(): Worker with active job excluded
   - testMatchWorkers_sortsByRatingThenDistance(): verify ordering
   - testMatchWorkers_noWorkersFound(): returns empty list, job updated accordingly

4. StateMachineTest.java:
   Test all valid and invalid transitions:
   - testTransition_requested_to_pendingDeposit_bySystem(): valid
   - testTransition_confirmed_to_inProgress_byWorker(): valid
   - testTransition_confirmed_to_inProgress_byRequester_throws(): 403
   - testTransition_complete_to_disputed_within2hr(): valid
   - testTransition_complete_to_disputed_after2hr_throws(): invalid
   - testTransition_cancelled_to_anything_throws(): terminal state
   (At least 10 test cases total)

Provide complete file contents.

P1-23 — Production Deployment

Phase: 1 — MVP | **Week:** W17 | **Depends on:** All P1-01 through P1-22

Dependencies

- All P1 tasks complete and passing tests
- SendGrid domain verification complete (set up in Week 1 per R6 risk mitigation)
- All GCP Secret Manager secrets populated with production values
- Firebase Hosting configured for production Firebase project

Tools & Libraries

Tool / Library	Version	Purpose
gcloud CLI	Latest	Cloud Run deployment
Firebase CLI	Latest	Hosting deployment
Stripe CLI	Latest	Webhook endpoint verification

Assumptions

- Blue/green deployment via Cloud Run traffic splitting

- All production secrets are in GCP Secret Manager (not in application.yml or environment)
- Smoke tests are manual; automated smoke tests are a post-MVP improvement

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Produce the production deployment guide and final configuration (P1-23).

Files to create/update:
  backend/src/main/resources/application-prod.yml (final)
  docs/deployment-guide.md

1. application-prod.yml (final production configuration):
   logging.level.com.snowreach: INFO
   logging.level.root: WARN
   server.tomcat.max-threads: 200
   spring.quartz.job-store-type: memory
   snowreach.firebase.use-emulator: false
   management.endpoint.health.show-details: never

2. docs/deployment-guide.md with these sections:

   PRE-DEPLOYMENT CHECKLIST:
   - [ ] All P1-22 integration tests passing
   - [ ] SendGrid domain verified (SPF, DKIM, DMARC DNS records live)
   - [ ] Stripe webhooks endpoint registered at https://api.snowreach.ca/webhooks/stripe
   - [ ] All GCP Secret Manager secrets populated (list each one)
   - [ ] Firebase security rules deployed: firebase deploy --only firestore:rules,storage
   - [ ] Firestore indexes deployed: firebase deploy --only firestore:indexes
   - [ ] CORS config updated with production frontend URL

   BACKEND DEPLOYMENT:
   gcloud builds submit backend/ --tag gcr.io/PROJECT_ID/snowreach-api:v1.0.0
   gcloud run deploy snowreach-api --image gcr.io/PROJECT_ID/snowreach-api:v1.0.0 --region northamer
   (Route 10% traffic, monitor for 30 min, then route 100%)
   gcloud run services update-traffic snowreach-api --to-latest

   FRONTEND DEPLOYMENT:
   cd frontend && npm run build
   cd ../firebase && firebase use prod && firebase deploy --only hosting

   SMOKE TEST CHECKLIST (manual):
   - [ ] GET https://api.snowreach.ca/api/health returns 200

```
- [ ] Firebase Auth: register test user, login, get ID token
- [ ] POST /api/users with token returns 201
- [ ] POST /api/jobs returns 201 with REQUESTED status
- [ ] Stripe CLI: stripe listen --forward-to https://api.snowreach.ca/webhooks/stripe
      stripe trigger payment_intent.succeeded -- verify job transitions to CONFIRMED
- [ ] SendGrid: send test email, verify delivery (check inbox, not spam)
- [ ] Firebase Hosting: visit https://snowreach-prod.web.app, verify React app loads

ROLLBACK PROCEDURE:
gcloud run services update-traffic snowreach-api --to-revisions PREVIOUS=100
```

Provide complete file contents.

P2-01 — Dispute Workflow API

Phase: 2 — Structured Disputes | **Week:** W18-19 | **Depends on:** P1-13, P1-10

Dependencies

- P1-13 complete (state machine with DISPUTED transition)
- P1-10 complete (Quartz scheduler working)
- P1-18 complete (notifications wired)

Tools & Libraries

Tool / Library	Version	Purpose
Quartz Scheduler	2.3.x	2-hour dispute window timer
firebase-admin	9.3.x	Dispute document management

Assumptions

- The 2-hour dispute window starts when job transitions to COMPLETE
- A Quartz timer is set at COMPLETE that, when it fires, prevents any new DISPUTED transitions
- The dispute window is enforced in the state machine transition() guard (already coded in P1-13)
- Only the Requester can open a dispute (Worker can dispute via Admin if needed)

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement the full dispute workflow API (P2-01).

Files:
  backend/src/main/java/com/snowreach/controller/DisputeController.java
  backend/src/main/java/com/snowreach/service/DisputeService.java
  backend/src/main/java/com/snowreach/model/Dispute.java
  backend/src/main/java/com/snowreach/dto/DisputeRequest.java
  backend/src/main/java/com/snowreach/scheduler/DisputeTimerJob.java

Firestore schema for disputes/{disputeId}:
  disputeId (UUID), jobId, openedByUid, openedAt (Timestamp),
  status (OPEN | RESOLVED), resolution (RELEASED | REFUNDED | SPLIT),
  splitPercentageToWorker (int, 0-100),
  requesterStatement (String), workerStatement (String),
  evidenceUrls (List<String>), adminNotes (String),
  resolvedByAdminUid, resolvedAt (Timestamp)

1. DisputeRequest.java:
   @NotBlank String statement (the opener's account of what happened)

2. DisputeService.java:
   openDispute(String jobId, String requesterId, DisputeRequest req) -> Dispute:
   a. Read job. Validate: status == COMPLETE, requesterId == job.requesterId.
   b. Check job.disputedAt is null (not already disputed).
   c. Check job.completedAt within last 2 hours:
      if (Instant.now().isAfter(job.completedAt.toInstant().plusSeconds(7200)))
        throw InvalidTransitionException("Dispute window has closed.")
   d. Create dispute document with status=OPEN, requesterStatement=req.statement.
   e. Call jobService.transition(jobId, "DISPUTED", requesterId, "Dispute opened").
   f. Update job: disputeId = new disputeId.
   g. Notify both parties and Admin (notificationService).
   h. Audit log.
   i. Return dispute.

   resolveDispute(String disputeId, String adminUid, String resolution,
                  int splitPct, String adminNotes) -> Dispute:
   a. Read dispute. Validate status == OPEN, adminUid is ADMIN.
   b. Update dispute: status=RESOLVED, resolution, splitPercentageToWorker, adminNotes,
      resolvedByAdminUid, resolvedAt.
   c. Based on resolution:
      RELEASED: call paymentService.releasePayment(jobId).
      REFUNDED: call paymentService.refundJob(jobId).
      SPLIT: call paymentService.splitPayment(jobId, splitPct) -- implement in PaymentService.
   d. Transition job: RELEASED->RELEASED or REFUNDED based on resolution.
   e. Notify both parties of outcome.

f. Audit log.
g. Return dispute.

3. DisputeController.java:
POST /api/jobs/{jobId}/dispute (REQUESTER role):
Calls disputeService.openDispute(). Returns 201.
GET /api/disputes/{disputeId} (parties to the job or ADMIN)
POST /api/disputes/{disputeId}/statement (party to dispute, add/update statement)
POST /api/disputes/{disputeId}/resolve (ADMIN only):
Body: { resolution, splitPercentageToWorker, adminNotes }

4. DisputeTimerJob.java (Quartz):
Scheduled when job transitions to COMPLETE (add to side effects in JobService.transition()).
Fires after 2 hours. In execute(): log "Dispute window closed for job {jobId}".
The actual enforcement is in the state machine guard in transition().
This job is just for logging/future extension (e.g. auto-release if no dispute).

Provide complete file contents.

P2-02 — Dispute Evidence Upload

Phase: 2 — Structured Disputes | **Week:** W18 | **Depends on:** P2-01, P1-15

Dependencies

- P2-01 complete (dispute document exists)
- P1-15 complete (StorageService pattern established)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firebase Storage upload
spring-boot-starter-web	3.2.x	Multipart handling

Assumptions

- Maximum 5 files per party (Requester and Worker have separate limits)
- Accepted file types: JPEG, PNG, PDF
- Maximum 20 MB per file
- Files stored at disputes/{disputeId}/{party}/{uuid}.{ext} where party = requester or worker

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement dispute evidence file upload (P2-02).

Add to:
backend/src/main/java/com/snowreach/service/StorageService.java
backend/src/main/java/com/snowreach/controller/DisputeController.java

1. StorageService additions:
uploadDisputeEvidence(String disputeId, String partyRole, MultipartFile file) -> String:
a. Validate MIME: image/jpeg, image/png, application/pdf only.
b. Validate size: <= 20 MB.
c. Generate path: "disputes/" + disputeId + "/" + partyRole.toLowerCase() + "/" + UUID + ext.
d. Upload to Firebase Storage via Admin SDK.
e. Return download URL.

2. DisputeController additions:
POST /api/disputes/{disputeId}/evidence (REQUESTER or WORKER, must be party to dispute):
a. Determine partyRole: REQUESTER if actorUid == job.requesterId, else WORKER.
b. Count existing evidence for this party (filter evidenceUrls by storage path prefix).
If count >= 5: return 400 "Maximum 5 evidence files per party."
c. Call storageService.uploadDisputeEvidence().
d. Atomically append URL to dispute.evidenceUrls in Firestore.
e. Return { url, totalEvidenceCount }.

Provide complete file additions (delta only, not full files).

P2-03 — SnowReach Assurance Adjudication

Phase: 2 — Structured Disputes | **Week:** W19-20 | **Depends on:** P2-01, P2-02

Dependencies

- P2-01 complete (resolveDispute method exists)
- P1-12 complete (PaymentService with refund and transfer)

Tools & Libraries

Tool / Library	Version	Purpose
stripe-java	25.x	Split payment (partial transfer + partial refund)

Assumptions

- SPLIT resolution: platform first captures full amount, transfers worker's share, refunds remainder
- Minimum worker split: 0% (full refund to Requester); maximum: 100% (full release to Worker)
- Admin cannot resolve a dispute they did not review (future enhancement; not enforced in Phase 2)

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Implement split payment for SnowReach Assurance dispute resolution (P2-03).

Add to: backend/src/main/java/com/snowreach/service/PaymentService.java

splitPayment(String jobId, int workerPercentage) -> void:
  workerPercentage: 0-100 integer (e.g. 75 means Worker gets 75% of netWorkerCents).
  a. Read job: netWorkerCents, hstCents, depositAmountCents, currentWorkerId, stripePaymentIntentId.
  b. Confirm-capture the PaymentIntent for the full deposit amount (if not already captured).
  c. workerAmountCents = round(netWorkerCents * workerPercentage / 100.0) + hstCents
     (HST always flows to Worker regardless of split).
  d. requesterRefundCents = depositAmountCents - workerAmountCents.
  e. If workerAmountCents > 0:
     Create Stripe Transfer to worker's Connected account for workerAmountCents.
  f. If requesterRefundCents > 0:
     Create Stripe Refund on the PaymentIntent for requesterRefundCents.
  g. Update job: stripeTransferId, stripeRefundId.
  h. Audit log both actions.
  i. Notify both parties of amounts.

Also ensure resolveDispute in DisputeService calls the correct PaymentService method:
  RELEASED -> releasePayment(jobId) -- full to worker
  REFUNDED -> refundJob(jobId) -- full to requester
  SPLIT -> splitPayment(jobId, splitPercentageToWorker)

Provide complete delta (only the new splitPayment method and the resolveDispute wiring).
```

P2-04 — Admin Dispute Resolution UI

Phase: 2 — Structured Disputes | **Week:** W20 | **Depends on:** P2-01 through P2-03

Dependencies

- P2-01, P2-02, P2-03 complete (dispute API fully working)
- P0-14 complete (Admin JobDetail React page exists)

Tools & Libraries

Tool / Library	Version	Purpose
React	18.x	Component updates
Axios (via api.js)	1.x	Fetch dispute data

Assumptions

- The dispute resolution UI is embedded in the existing Admin JobDetail page (not a separate route)
- Evidence is displayed as a photo gallery (images) and download links (PDFs)
- Resolution form is only shown when dispute status is OPEN

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Update the Admin Job Detail page with a full dispute resolution UI (P2-04).

Update:
  frontend/src/pages/admin/JobDetail.jsx
  frontend/src/services/api.js (add dispute API calls)

1. api.js additions:
   getDispute(disputeId), resolveDispute(disputeId, body), uploadEvidence(disputeId, formData)
   addStatement(disputeId, statement)

2. JobDetail.jsx dispute section update (when job.status == DISPUTED):
   a. Fetch dispute document on component mount.
   b. Evidence gallery:
      - Photos: render <img> thumbnails in a grid. Click to view full-size in a lightbox Modal.
      - PDFs: render as download link with PDF icon.
      - Label each piece of evidence: "Requester" or "Worker" based on storage path.
   c. Statements:
      Requester statement in a Card with blue left border.
      Worker statement in a Card with purple left border.
      If a party has not submitted a statement: show "No statement submitted."
   d. Resolution form (shown when dispute.status == OPEN):

Radio group: "Release to Worker" | "Refund Requester" | "Split"
If Split selected: show a range slider 0-100 with labels "0% to Worker" and "100% to Worker".
Show calculated amounts live: "Worker receives: $XX | Requester refund: $XX"
Textarea: Admin resolution notes (required, min 20 chars).
Button "Resolve Dispute" -> confirm Modal ("This action is irreversible. Confirm?")
-> on confirm: call api.resolveDispute() -> show success banner.

e. After resolution: dispute section shows "Resolved" badge, resolution details read-only.

Provide complete updated JobDetail.jsx.

P2-05 — Worker Concurrent Capacity

Phase: 2 — Structured Disputes | **Week:** W21 | **Depends on:** P1-09, P1-06

Dependencies

- P1-09 complete (MatchingService filters busy Workers)
- P1-06 complete (Worker profile has maxConcurrentJobs field)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Firestore query for active job count

Assumptions

- In Phase 1, maxConcurrentJobs was effectively 1 (single busy-check)
- In Phase 2, a Worker with maxConcurrentJobs=2 can accept a second job while first is IN_PROGRESS
- Minimum rating of 4.0 required to set maxConcurrentJobs > 1 (enforced on PATCH request)
- The MatchingService update counts IN_PROGRESS jobs against the Worker's concurrent limit

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement Worker concurrent job capacity (P2-05).

Update:
backend/src/main/java/com/snowreach/service/MatchingService.java

backend/src/main/java/com/snowreach/controller/WorkerController.java

1. MatchingService update:
   In matchAndStoreWorkers(), replace the binary "is busy" check with a capacity check:
   a. Query jobs where currentWorkerId == workerUid AND status IN [CONFIRMED, IN_PROGRESS].
   b. Count results: activeJobCount.
   c. Skip Worker if activeJobCount >= worker.maxConcurrentJobs.
   This allows Workers with maxConcurrentJobs=2 to take a second job.

2. WorkerController addition:
   PATCH /api/workers/{uid}/capacity (WORKER own uid or ADMIN):
   Body: { maxConcurrentJobs: int (1-3) }
   Validation:
   - maxConcurrentJobs must be 1-3.
   - If setting > 1: Worker must have averageRating >= 4.0 and totalJobsCompleted >= 10.
   - If validation fails: return 400 with descriptive message.
   On success: update workers/{uid}.maxConcurrentJobs.
   Audit log the change.

Provide complete delta.

P2-06 — Analytics Data Pipeline

Phase: 2 — Structured Disputes | **Week:** W22-23 | **Depends on:** P1-20, P1-10

Dependencies

- P1 complete (all Firestore collections populated in production)
- P1-03 complete (Quartz available for scheduled jobs)

Tools & Libraries

Tool / Library	Version	Purpose
Quartz Scheduler	2.3.x	Daily aggregation job
firebase-admin	9.3.x	Firestore read + write

Assumptions

- Aggregation reads all jobs, users, ratings for the previous calendar day
- Writes summary to Firestore analytics/daily/{YYYY-MM-DD} and updates analytics/summary/current
- 90-day rolling window: documents older than 90 days are deleted
- The pipeline runs at 3 AM daily (after AuditIntegrityJob at 2 AM)

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement the daily analytics data pipeline (P2-06).

Files:
  backend/src/main/java/com/snowreach/scheduler/AnalyticsJob.java
  backend/src/main/java/com/snowreach/service/AnalyticsService.java

Firestore schema for analytics/daily/{YYYY-MM-DD}:
  date, jobsCompleted (int), jobsCancelled (int), jobsDisputed (int),
  grossRevenueCents (long), platformRevenueCents (long), workerPayoutsCents (long),
  hstCollectedCents (long), avgRating (double), newWorkers (int), newRequesters (int),
  cancellationRate (double), disputeRate (double)

Firestore doc analytics/summary/current:
  totalJobsAllTime, totalGrossRevenueCents, totalWorkerPayoutsCents,
  totalPlatformRevenueCents, totalWorkers, totalRequesters,
  overallAverageRating, lastUpdated

1. AnalyticsService.java:
   computeDailyStats(LocalDate date) -> Map<String, Object>:
   a. Define day boundaries: start = date.atStartOfDay(ZoneId.of("America/Toronto")),
      end = date.plusDays(1).atStartOfDay(same zone).
   b. Query jobs collection: filter completedAt >= start AND completedAt < end.
      For each: accumulate grossRevenueCents, platformFeeCents, workerNetCents, hstCents.
      Count COMPLETE, CANCELLED, DISPUTED statuses.
   c. Query ratings for the day: compute avgRating.
   d. Query users created in date range: count by role.
   e. Write to analytics/daily/{date}.
   f. Update analytics/summary/current (Firestore transaction to increment totals).

   cleanupOldDailyStats() -> void:
   Delete analytics/daily/* documents where date < 90 days ago.

2. AnalyticsJob.java (Quartz, runs at 3 AM daily):
   Compute stats for LocalDate.now(ZoneId.of("America/Toronto")).minusDays(1).
   Call analyticsService.computeDailyStats(yesterday).
   Call analyticsService.cleanupOldDailyStats().
   Log: "Analytics pipeline complete for {date}: {jobsCompleted} jobs, ${revenue}."

Provide complete file contents.

P2-07 — Analytics Dashboard

Phase: 2 — Structured Disputes | **Week:** W23-24 | **Depends on:** P2-06

Dependencies

- P2-06 complete (analytics collection populated daily)
- P0-13 complete (Admin Dashboard exists with tab structure)

Tools & Libraries

Tool / Library	Version	Purpose
Chart.js	4.x	Charts via npm
react-chartjs-2	5.x	React wrapper for Chart.js
Axios (api.js)	1.x	Fetch analytics data

Assumptions

- Analytics page is a new route: /admin/analytics (already in router from P0-01)
- Date range picker uses native HTML date inputs (no third-party date picker library)
- Chart.js is installed via npm, not CDN

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Build the Analytics Dashboard (P2-07).

Files:
  frontend/src/pages/admin/Analytics.jsx + Analytics.module.css
  backend/src/main/java/com/snowreach/controller/AdminController.java (analytics endpoint)

1. AdminController addition:
   GET /api/admin/analytics?from=YYYY-MM-DD&to=YYYY-MM-DD (ADMIN only):
   Read analytics/daily/* documents within the date range (from and to inclusive).
   Return array of daily stat objects ordered by date.
   Also return analytics/summary/current for all-time totals.

2. Analytics.jsx:
   Date range selector: two date inputs (from, to). Default: last 30 days.
   Fetch button (or auto-fetch on date change).

CHARTS (use react-chartjs-2):
a. Line chart: X=date, Y1=jobsCompleted (blue), Y2=grossRevenueCents/100 (green).
Dual y-axis. Title: "Jobs & Revenue by Day."
b. Bar chart: X=week, stacked bars: completed (green) vs cancelled (red) vs disputed (amber).
Title: "Job Outcomes by Week."
c. Pie chart: current totals breakdown: platform revenue, worker payouts, HST.
Title: "Revenue Distribution (All Time)."

ALL-TIME STATS CARDS (3 cards):
Total Jobs, Total Gross Revenue, Overall Average Rating.

TOP WORKERS TABLE: (separate API call: GET /api/admin/workers?sort=jobs&size=10)
Columns: Name, Jobs Completed, Average Rating, Total Earned.

EXPORT BUTTON: "Export CSV"
On click: GET /api/admin/reports/transactions?from=...&to=... (implemented in P3-07).
Trigger browser download of the CSV.

Provide complete file contents for all files.

P2-08 — Platform Health Monitoring

Phase: 2 — Structured Disputes | **Week:** W25 | **Depends on:** P1-23

Dependencies

- P1-23 complete (production deployed to Cloud Run)
- GCP project with billing enabled

Tools & Libraries

Tool / Library	Version	Purpose
GCP Cloud Monitoring	N/A	Uptime checks, alert policies
Spring Actuator	3.2.x	Extended health endpoint

Assumptions

- Monitoring is configured via gcloud CLI and GCP Console
- Spring Actuator /api/health endpoint provides custom health checks
- Alert emails go to the admin email address

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement platform health monitoring (P2-08).

Files to create:
  docs/runbook.md
  backend/src/main/java/com/snowreach/config/HealthConfig.java

1. HealthConfig.java:
   Implement custom Spring Boot HealthIndicator beans:

   FirebaseHealthIndicator: attempts a Firestore read on a test document.
   Returns UP if successful, DOWN with detail if exception.

   QuartzHealthIndicator: checks that the Quartz Scheduler is started and
   not in standby mode. Returns UP or DOWN.

   Add to application.yml:
   management.health.firebase.enabled: true
   management.health.quartz.enabled: true
   management.endpoint.health.show-components: always

2. GCP Monitoring setup instructions (in runbook.md as gcloud commands):

   Uptime check (every 60s, 3-location majority):
   gcloud monitoring uptime-checks create https snowreach-api-health --resource-type=uptime-url --hos

   Alert policy: error rate > 1% over 5-minute window.
   Alert policy: p99 latency > 2000ms over 5-minute window.
   Alert policy: uptime check failure for 2+ consecutive checks.

   Notification channel: email to admin.

3. docs/runbook.md (complete on-call runbook):
   Sections:
   - Service overview (what SnowReach is, critical paths)
   - Alert descriptions and triage steps for each alert type
   - How to check Cloud Run logs: gcloud run services logs read snowreach-api
   - How to check Firestore emulator vs production (connection check)
   - How to manually trigger a payment release (via Admin dashboard)
   - How to roll back to previous Cloud Run revision
   - Escalation contacts and procedures

Provide complete file contents.

P3-01 — Certn Background Check API Integration

Phase: 3 — Trust & Safety | **Week:** W26-27 | **Depends on:** P1-06, P1-17

Dependencies

- P1-06 complete (Worker profile has backgroundCheckStatus field)
- P1-17 complete (email notifications working)
- Certn account created and API key obtained (start application at end of Phase 2)

Tools & Libraries

Tool / Library	Version	Purpose
spring-boot-starter-web	3.2.x	HTTP client for Certn API
spring-boot-starter-validation	3.2.x	Webhook payload validation

Assumptions

- Certn API is a REST API; use Spring RestTemplate or WebClient to call it
- Certn sends a webhook when the check result is ready
- backgroundCheckStatus field values: PENDING, SUBMITTED, CLEAR, CONSIDER, SUSPENDED, REJECTED
- CERTN_API_KEY is stored in GCP Secret Manager

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Integrate Certn background check API (P3-01).

```
Files:
  backend/src/main/java/com/snowreach/service/BackgroundCheckService.java
  backend/src/main/java/com/snowreach/controller/WebhookController.java (additions)
  backend/src/main/java/com/snowreach/controller/WorkerController.java (additions)

1. BackgroundCheckService.java:
   submitBackgroundCheck(String workerUid) -> String (Certn orderId):
   a. Read Worker profile (name, DOB, address from user/worker docs).
   b. POST to Certn API: https://api.certn.co/v1/applicants/
      Payload: { first_name, last_name, date_of_birth, email, package: "criminal_record" }
```

```
    Authorization: Token CERTN_API_KEY
  c. Parse response: extract applicant_id / order_id.
  d. Update workers/{uid}: certnOrderId = orderId, backgroundCheckStatus = SUBMITTED.
  e. Audit log.
  f. Return orderId.

  handleCertnWebhook(String orderId, String status, String rawPayload) -> void:
  a. Look up Worker by certnOrderId in Firestore (query workers where certnOrderId == orderId).
  b. Map Certn status to backgroundCheckStatus:
    "PASS" -> CLEAR; "REVIEW" -> CONSIDER; "FAIL" -> SUSPENDED
  c. Update workers/{uid}.backgroundCheckStatus.
  d. If CLEAR: call setActive(uid, true); send "You're approved!" email.
  e. If CONSIDER: add to adminReviewQueue collection; notify Admin.
  f. If SUSPENDED: ensure isActive=false; notify Worker; notify Admin.
  g. Audit log.

2. WebhookController additions:
  POST /webhooks/certn (no auth -- verify by HMAC or shared secret if Certn supports it):
  Parse body: { orderId, status, ... }.
  Call backgroundCheckService.handleCertnWebhook().
  Return 200 OK.

3. WorkerController addition:
  POST /api/workers/{uid}/background-check (WORKER own uid):
  Trigger background check submission. Worker must consent (accept body: { consented: true }).
  Call backgroundCheckService.submitBackgroundCheck().

Provide complete file contents.
```

P3-02 — Background Check Status Flow

Phase: 3 — Trust & Safety | **Week:** W27 | **Depends on:** P3-01

Dependencies

- P3-01 complete (backgroundCheckStatus updated by webhook)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	adminReviewQueue collection

Assumptions

- Admin override is needed when Certn returns CONSIDER (ambiguous result)
- Admin can either approve (set CLEAR) or reject (set REJECTED) after manual review
- Worker unlock (isActive=true) happens automatically on CLEAR only

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Implement background check status tracking and Admin review flow (P3-02).

Add to:
  backend/src/main/java/com/snowreach/controller/AdminController.java
  backend/src/main/java/com/snowreach/controller/WorkerController.java
  backend/src/main/java/com/snowreach/service/BackgroundCheckService.java

1. BackgroundCheckService additions:
   adminOverride(String workerUid, String decision, String adminUid, String reason):
   decision = "APPROVED" | "REJECTED"
   If APPROVED: set backgroundCheckStatus=CLEAR, isActive=true, send approval email.
   If REJECTED: set backgroundCheckStatus=REJECTED, isActive=false, send rejection email.
   Remove from adminReviewQueue.
   Audit log with adminUid and reason.

2. AdminController additions:
   GET /api/admin/review-queue (ADMIN only):
   Read adminReviewQueue collection. Return list of items with workerUid, certnStatus,
   submittedAt, workerName, workerEmail.

   POST /api/admin/workers/{uid}/background-check-decision (ADMIN only):
   Body: { decision: "APPROVED" | "REJECTED", reason: string }
   Calls backgroundCheckService.adminOverride().

3. WorkerController addition:
   GET /api/workers/{uid}/background-check-status (WORKER own uid or ADMIN):
   Returns: { backgroundCheckStatus, certnOrderId, submittedAt }

Provide complete delta.
```

P3-03 — Insurance Declaration

Phase: 3 — Trust & Safety | **Week:** W28 | **Depends on:** P1-15, P1-06

Dependencies

- P1-15 complete (StorageService for file upload)
- P1-06 complete (Worker profile with insuranceStatus field)

Tools & Libraries

Tool / Library	Version	Purpose
Quartz Scheduler	2.3.x	Annual renewal reminder job
firebase-admin	9.3.x	Update Worker insuranceStatus
sendgrid-java	4.x	Renewal reminder email

Assumptions

- Insurance status values: NONE, PENDING_REVIEW, VALID, EXPIRING_SOON, EXPIRED
- Admin must manually verify insurance document and set status to VALID (Phase 3 — no auto-verification)
- Annual renewal reminder fires 30 days before expiry
- If expired, Worker isActive is set to false

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement insurance declaration upload and renewal management (P3-03).

Files:
  backend/src/main/java/com/snowreach/controller/WorkerController.java (additions)
  backend/src/main/java/com/snowreach/service/InsuranceService.java
  backend/src/main/java/com/snowreach/scheduler/InsuranceRenewalJob.java

1. InsuranceService.java:
   uploadInsuranceDoc(String workerUid, MultipartFile file, LocalDate expiryDate) -> void:
   a. Validate: file is PDF (application/pdf), size <= 20 MB.
   b. Upload to Firebase Storage: "workers/{uid}/insurance/{UUID}.pdf"
   c. Update Worker: insuranceDocUrl, insuranceExpiryDate, insuranceStatus=PENDING_REVIEW.
   d. Audit log.
   e. Notify Admin: "Worker {name} submitted insurance document for review."

adminVerifyInsurance(String workerUid, boolean approved, String adminUid) -> void:
If approved: set insuranceStatus=VALID. If !approved: set insuranceStatus=NONE, delete doc URL.
Audit log. Notify Worker of decision.

2. WorkerController additions:
POST /api/workers/{uid}/insurance (WORKER own uid):
Multipart form: file + expiryDate (ISO-8601 date string).
Calls insuranceService.uploadInsuranceDoc().
POST /api/admin/workers/{uid}/insurance-verify (ADMIN only):
Body: { approved: boolean }.
Calls insuranceService.adminVerifyInsurance().

3. InsuranceRenewalJob.java (Quartz, runs daily at 4 AM):
Query workers where insuranceStatus in [VALID, EXPIRING_SOON].
For each:
a. If expiryDate < today: set insuranceStatus=EXPIRED, isActive=false.
Send "Your insurance has expired." email.
b. Else if expiryDate <= today + 30 days: set insuranceStatus=EXPIRING_SOON.
Send renewal reminder email (only once per 7 days -- check lastReminderSent field).

Provide complete file contents.

P3-04 — Trust Badge System

Phase: 3 — Trust & Safety | **Week:** W29 | **Depends on:** P3-01, P3-03, P1-16

Dependencies

- P3-01 complete (backgroundCheckStatus: CLEAR)
- P3-03 complete (insuranceStatus: VALID)
- P1-16 complete (averageRating and totalJobsCompleted updated)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Update workers/{uid}/badges subcollection

Assumptions

- Badges are stored in a subcollection workers/{uid}/badges/{badgeId}
- Badge eligibility is re-evaluated whenever relevant Worker fields change
- Badges can be manually granted or revoked by Admin (audit logged)
- React displays badge icons in Worker profile modal and search results

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement the Trust Badge system (P3-04).

Files:
  backend/src/main/java/com/snowreach/service/BadgeService.java
  backend/src/main/java/com/snowreach/controller/AdminController.java (additions)
  frontend/src/pages/requester/WorkerProfile.jsx (badge display update)

Badge types and eligibility:
  VERIFIED: backgroundCheckStatus == CLEAR
  INSURED: insuranceStatus == VALID
  TOP_RATED: averageRating >= 4.8 AND totalJobsCompleted >= 25
  EXPERIENCED: totalJobsCompleted >= 100

Firestore schema workers/{uid}/badges/{badgeId}:
  badgeId (type), awardedAt, awardedBySystem (boolean), awardedByAdminUid,
  isActive (boolean), revokedAt, revokedByAdminUid, revokedReason

1. BadgeService.java:
   evaluateBadges(String workerUid) -> void:
   a. Read Worker document.
   b. For each badge type: check eligibility condition.
   c. Read existing badge document.
   d. If eligible and no active badge: create badge (awardedBySystem=true, isActive=true).
   e. If not eligible and active badge exists: set isActive=false, revokedAt=now.
   f. Audit log changes.

   Call evaluateBadges() from:
   - WorkerService.updateAverageRating() after updating rating
   - BackgroundCheckService.handleCertnWebhook() when status becomes CLEAR
   - InsuranceService.adminVerifyInsurance() when approved

   adminGrantBadge(String workerUid, String badgeType, String adminUid) -> void:
   adminRevokeBadge(String workerUid, String badgeType, String adminUid, String reason) -> void:

2. AdminController additions:
   POST /api/admin/workers/{uid}/badges/{badgeType}/grant (ADMIN only)
   POST /api/admin/workers/{uid}/badges/{badgeType}/revoke (ADMIN only):
     Body: { reason: string }

3. WorkerProfile.jsx update:
   Fetch workers/{uid}/badges subcollection (via backend endpoint).
   Display active badges as icon chips with tooltips:

```
  VERIFIED: blue shield + checkmark, tooltip "Background check passed"
  INSURED: green umbrella, tooltip "Liability insurance on file"
  TOP_RATED: gold star, tooltip "4.8+ rating, 25+ jobs"
  EXPERIENCED: silver badge, tooltip "100+ jobs completed"
```

Provide complete file contents.

P3-05 — Fraud Detection Rules Engine

Phase: 3 — Trust & Safety | **Week:** W30-31 | **Depends on:** P1-12, P1-08

Dependencies

- P1-12 complete (releasePayment triggers are in place)
- P1-08 complete (jobs collection populated)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	fraudFlags collection

Assumptions

- Fraud rules are evaluated synchronously before every payout trigger
- If a flag is raised, the payout is paused (not cancelled) until Admin reviews
- Workers receive “payment under review” email; Admin receives alert
- Rules are hardcoded (no rules engine UI in Phase 3)

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Implement the fraud detection rules engine (P3-05).

Files:
  backend/src/main/java/com/snowreach/service/FraudDetectionService.java
  backend/src/main/java/com/snowreach/controller/AdminController.java (additions)

Firestore schema for fraudFlags/{flagId}:
  flagId (UUID), workerUid, jobId, ruleTriggered, detectedAt,
  status (PENDING_REVIEW | APPROVED | REJECTED), payoutAmountCents,
  reviewedByAdminUid, reviewedAt, reviewNotes
```

1. FraudDetectionService.java:
   checkBeforePayout(String jobId) -> boolean isSafe:
   a. Read job: currentWorkerId, netWorkerCents, createdAt.
   b. Read Worker: createdAt (account age).
   c. Run all rules:

   RULE 1 — Velocity check:
   Query jobs where currentWorkerId == workerUid AND status == COMPLETE
   AND completedAt >= now - 24 hours. If count > 5: flag.

   RULE 2 — Large payout:
   If netWorkerCents > 50000 (> $500 CAD): flag.

   RULE 3 — Rating manipulation:
   Read last 10 ratings for Worker. If averageRating jumped > 1.0 in last 10 jobs: flag.
   (Simplified check: compare current averageRating to rating 10 jobs ago.)

   RULE 4 — New account large payout:
   If account age < 7 days AND netWorkerCents > 20000: flag.

   If any rule triggers:
   - Create fraudFlags/{UUID} document.
   - Update job: payoutPaused = true.
   - Send Worker "payment under review" email.
   - Notify Admin.
   - Return false (not safe).
   If no rule triggers: return true.

2. Integrate into PaymentService.releasePayment():
   Before creating Stripe Transfer: call fraudDetectionService.checkBeforePayout(jobId).
   If returns false: throw PaymentException("Payout paused pending fraud review.").

3. AdminController additions:
   GET /api/admin/fraud-flags?status=PENDING_REVIEW (ADMIN only)
   POST /api/admin/fraud-flags/{flagId}/approve (ADMIN only):
     Set flag status=APPROVED, resume payout: call paymentService.releasePayment(jobId).
   POST /api/admin/fraud-flags/{flagId}/reject (ADMIN only):
     Set flag status=REJECTED, notify Worker payout denied, optionally refund Requester.

Provide complete file contents.

P3-06 — Enhanced Admin Controls

Phase: 3 — Trust & Safety | **Week:** W31 | **Depends on:** P1-19, P1-05

Dependencies

- P1-19 complete (Admin dashboard API working)
- P1-05 complete (user status field in Firestore)

Tools & Libraries

Tool / Library	Version	Purpose
firebase-admin	9.3.x	Revoke Firebase custom claims

Assumptions

- Ban is reversible (unban endpoint exists); Suspend is temporary with duration
- Banning a user cancels all their open jobs and issues refunds
- Firebase custom claims are revoked on ban so existing tokens become invalid on next verification
- All actions are audit logged with actorUid and reason

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Implement enhanced Admin controls for user management (P3-06).

Add to:
  backend/src/main/java/com/snowreach/controller/AdminController.java
  backend/src/main/java/com/snowreach/service/UserService.java

1. UserService additions:
   banUser(String uid, String adminUid, String reason) -> void:
   a. Set users/{uid}.status = BANNED.
   b. Revoke Firebase refresh tokens: FirebaseAuth.getInstance().revokeRefreshTokens(uid).
   c. Set Firebase custom claim roles to [] (empty) to invalidate future tokens.
   d. Find all open jobs for this user (as Requester or Worker).
      For each: cancel the job and process refund.
   e. Send ban notification email.
   f. Audit log.

   suspendUser(String uid, String adminUid, String reason, int durationDays) -> void:

Set status=SUSPENDED, suspendedUntil=now+durationDays.
Revoke tokens. Schedule Quartz job to auto-unsuspend at suspendedUntil.
Audit log.

unbanUser(String uid, String adminUid, String reason) -> void:
Set status=ACTIVE. Restore Firebase custom claims from roles[] in Firestore.
Audit log.

2. AdminController additions:
POST /api/admin/users/{uid}/ban (ADMIN): Body: { reason }
POST /api/admin/users/{uid}/unban (ADMIN): Body: { reason }
POST /api/admin/users/{uid}/suspend (ADMIN): Body: { reason, durationDays }
POST /api/admin/jobs/bulk-action (ADMIN):
Body: { jobIds: [...], action: "release" | "refund" }
Process each job sequentially; return summary { succeeded, failed }.

3. Frontend Admin Dashboard update (Dashboard.jsx):
In the User Registrations tab, add action buttons per user row:
"Ban", "Suspend", "View Profile" -- each opens a confirm Modal.
After action: refresh the user list.

Provide complete file contents.

P3-07 — Compliance Reporting

Phase: 3 — Trust & Safety | **Week:** W32 | **Depends on:** P1-19, P2-07

Dependencies

- P1-19 complete (AdminController exists)
- P2-07 complete (Analytics dashboard has export button)

Tools & Libraries

Tool / Library	Version	Purpose
Apache Commons CSV	1.10.x	CSV generation
Spring rate limiting	Custom or Bucket4j	10 requests/hour throttle

Assumptions

- Export endpoint is Admin-only and rate-limited

- CSV includes all completed/cancelled/refunded jobs in the date range
- JSON export is the same data as JSON array
- Every export is audit logged

AI Implementation Prompt

```
[PREPEND CONTEXT PREAMBLE]

TASK: Implement compliance reporting and transaction export (P3-07).

Files:
  backend/src/main/java/com/snowreach/controller/AdminController.java (additions)
  backend/pom.xml (add commons-csv dependency)

1. pom.xml addition:
   <dependency>
     <groupId>org.apache.commons</groupId>
     <artifactId>commons-csv</artifactId>
     <version>1.10.0</version>
   </dependency>

2. AdminController additions:
   GET /api/admin/reports/transactions?from=YYYY-MM-DD&to=YYYY-MM-DD&format=csv|json
   (ADMIN only, rate-limited: track request count per admin uid in Firestore,
    max 10 per hour; return 429 if exceeded):

   a. Parse from and to as LocalDate. Validate: range <= 366 days.
   b. Query jobs: completedAt, cancelledAt, or refundedAt within the date range.
      Include statuses: COMPLETE, RELEASED, SETTLED, CANCELLED, REFUNDED.
   c. For each job build a row:
      jobId, date, requesterId, requesterEmail, workerId, workerEmail,
      status, serviceTypes, grossAmountCents, hstCents, platformFeeCents,
      workerNetCents, cancellationFeeCents (if applicable)
   d. Audit log: "REPORT_EXPORTED", actorUid, date range, row count.
   e. If format=csv:
      Build CSV using Apache Commons CSVFormat.EXCEL with header row.
      Return as ResponseEntity<byte[]> with headers:
      Content-Type: text/csv
      Content-Disposition: attachment; filename="snowreach-transactions-{from}-{to}.csv"
   f. If format=json:
      Return ResponseEntity<List<Map>> as JSON.

   GET /api/admin/reports/workers-summary?year=YYYY (ADMIN only):
   For each Worker: total jobs, total gross payout, total HST received.
   Useful for annual T4A-equivalent reporting.
   Return JSON array sorted by workerDisplayName.
```

Provide complete file additions.

P3-08 — Security Audit and Penetration Testing Checklist

Phase: 3 — Trust & Safety | **Week:** W33 | **Depends on:** All P3 tasks complete

Dependencies

- All phases complete (full system running in production)

Assumptions

- This is a process/documentation task; produces a checklist document
- Security audit may be internal or external (external recommended before public launch)
- Each checklist item includes a test method and expected result

AI Implementation Prompt

[PREPEND CONTEXT PREAMBLE]

TASK: Produce the security audit and penetration testing checklist (P3-08).

Create: docs/security-audit-checklist.md

The document should contain a comprehensive checklist organized into sections.
Each item format:
  - [ ] Item description | **Test method:** how to test it | **Expected:** what should happen

SECTIONS:

1. AUTHENTICATION AND AUTHORIZATION:
  - All API endpoints return 401 without valid Firebase ID token
  - Expired tokens are rejected (test with a token > 1 hour old)
  - /api/health and /webhooks/** accessible without auth
  - ADMIN endpoints return 403 when called with REQUESTER or WORKER token
  - WORKER endpoints return 403 when called with REQUESTER token
  - Banned user tokens are rejected (FirebaseAuth token revocation)
  - Custom claims (roles) match Firestore user document roles

2. DATA ACCESS CONTROL:
  - Requester cannot read another Requester's job (test: GET /api/jobs/{otherId})
  - Worker cannot read jobs they are not matched to

- Firestore security rules prevent direct client writes to all operational collections
- Firestore security rules allow notifications/{uid}/feed reads only for own uid

3. INPUT VALIDATION AND INJECTION:
- Job address field: test with SQL injection strings (should geocode-fail gracefully)
- File upload: test uploading a .exe renamed to .jpg (should reject by MIME type)
- File upload: test uploading a 15 MB image (should return 400)
- JSON body with extra fields: should be ignored (no mass assignment)
- specialNotes field: test with XSS payload <script>alert(1)</script> (should store as plain text, not executed

4. STRIPE SECURITY:
- Webhook without Stripe-Signature header returns 400
- Webhook with invalid signature returns 400
- Duplicate webhook event (same event ID) is idempotent (no double-charge)
- PaymentIntent amount cannot be overridden by client (amount computed server-side only)
- clientSecret is never logged or stored in Firestore

5. FILE UPLOAD SECURITY:
- Uploaded files cannot be executed (storage bucket has no execution permissions)
- Filenames are sanitized (original filename not preserved in storage path)
- Storage paths include UUID, not predictable sequential IDs

6. SECRETS AND CONFIGURATION:
- No API keys or secrets in Git history (use: git log -p | grep -i "key\|secret\|token")
- No secrets in Cloud Run environment variables visible via gcloud (use Secret Manager)
- Google Maps API key cannot be extracted from any client-side response or bundle
- CORS allows only the Firebase Hosting origin, not *

7. RATE LIMITING AND ABUSE:
- Compliance report export returns 429 after 10 requests/hour
- No unbounded Firestore queries (all queries have .limit() applied)

8. AUDIT LOG INTEGRITY:
- AuditIntegrityJob ran successfully for the last 7 days (check logs)
- Directly modifying an audit log document breaks the hash chain (detected by integrity job)
- Audit log contains entries for all state machine transitions tested

For each item, include: Test Method (how to test), Expected Result, Pass/Fail column.
Provide complete document content.